



Technische Universität München

Department of Mathematics



Master's Thesis

Scenario Trees Models vs. Lattice Models in Stochastic Optimization

Vadim Gorski

Supervisor: Prof. Dr. Claudia Klüppelberg

Advisor: Prof. Dr. David Wozabal

Submission Date: 15.03.2017

I assure the single handed composition of this Master's thesis only supported by declared resources.

Munich, 15.03.2017

Abstract

We compare the performance of tree-based models for solving stochastic programs to the performance of lattice-based approaches. We create a framework for such comparisons, argue why this framework allows us to make a definite statement about the relative performance and conduct various case studies. For the case studies, typical stochastic optimization problems from the contemporary literature are chosen. Our results show that especially for high-dimensional problems with increasing dependency between the stages, lattice-based solution methods offer a higher expected out-of-sample payoff, surpassing the payoff of the tree-based methods by up to 50%. Moreover, we use statistical testing methods to show that there is a significant difference in distributions of out-of-sample payoffs between those generated by a tree-based policy as compared to those generated by a lattice-based policy using Approximate Dual Dynamic Programming. We conclude that there is strong empirical evidence suggesting that the usage of lattice-based discretization together with the ADDP method provides the decision-maker with clear advantages.

Acknowledgements

I would like to sincerely thank my advisor Prof. David Wozabal for his continued support and guidance during the completion of this Thesis. I am especially thankful for his patience and for helping me through the many difficulties occurring along the way. I would also like to thank Prof. Claudia Klüppelberg who gave me the opportunity to work on this Thesis with Prof. Wozabal.

Being able to use the optimization library Quasar, developed by Prof. Nils Löhndorf, was an essential prerequisite for this Thesis and I'd like to thank him for the opportunity to use the library and for continuously providing updates and improvements.

Contents

1	Introduction	2
2	Stochastic programs	4
2.1	Two-stage stochastic programs	6
2.2	Multi-stage stochastic programs	6
2.3	Measures for evaluating solutions	7
2.4	Markov Decision Process (MDP)	9
3	Solution methods	11
3.1	Curse of dimensionality	11
3.2	Scenario trees and their construction	12
3.3	Solutions based on scenario trees	16
3.4	MDPs and scenario lattices	17
3.5	Solving MDPs and approximate dual dynamic programming	18
4	Analyzing and comparing solutions	21
4.1	Problem classes	21
4.2	Stochastic processes	22
4.3	Selecting models	31
4.4	Testing framework	31
4.5	Comparison criteria and statistical tests	33
4.6	Technical setup	38
5	Stochastic aggregate production planning (APP)	41
5.1	Problem definition	43
5.2	Modelling random demand for products	44
5.3	Numerical results	45
6	The flower girl optimization problem	47
6.1	Standard problem definition	48
6.2	Problem variations	49
6.3	Numerical results	50
7	Stochastic asset liability management problem (ALM)	53
7.1	Problem definition	55
7.2	Parameters and models for the stochastic processes	56
7.3	Numerical results	57

8	Conclusion and Outlook	59
9	Appendix	61
A	Coding examples	61
B	Generating NIG random variables	62

Chapter 1

Introduction

Along with a steady increase of computing power, optimization under uncertainty has become a very popular field of research. As a result, a variety of stochastic programming methods are now allowing users to improve their optimization models by incorporating randomness. While in some rare cases an analytical closed-form solution to a stochastic optimization problem might exist, most non-trivial problems require some sort of approximation, especially when the underlying random parameters' distributions are continuous. After establishing some basics of stochastic programming in *chapter 2*, we will discuss such approximations. One possible approximation can be represented by a so-called scenario-tree, which is a directed graph with vertices (or nodes) representing the possible states of a stochastic process and edges (or arcs) corresponding to the possible transitions from one node to another. Furthermore, all nodes are ordered according to their correspondence to a certain point in time, i.e. the first-stage node is followed by $n_1, \dots, n_T$ later-stage nodes for every stage $1, \dots, T$. However, these approximations are highly limited by the available computing power and therefore represent a trade-off between approximation precision and computational tractability. In order to ameliorate this difficulty, various methods for reducing scenario trees have been developed [20]. Further, the structure of the approximation itself can be changed to improve the discretization in some cases. An alternative structure is called scenario-lattice. Both approaches will be further discussed in *Chapter 3*.

Apart from the discretization, finding an optimal solution to the stochastic programming problem over an already discretized probability distribution presents another major obstacle. In case of discretizing the randomness via a scenario-tree, the deterministic equivalent over all scenario-tree nodes can be solved using standard linear programming or convex optimization methods. This will be discussed in detail in *Chapter 3.2*. For the scenario-lattice representation, an efficient approximate dynamic programming algorithm, the so-called ADDP (approximate dual dynamic programming) approach will be presented in *Chapter 3.5*.

Next, possible methods for analyzing and benchmarking the stochastic programming solutions will be presented in *Chapter 4*. This chapter also describes the numerical study conducted in this Thesis, along with the description of possible classes of stochastic programming models and the selection of appropriate models for conducting the study.

In the second part of the thesis, three case studies (*Chapters 5, 6, 7*) based on representative stochastic programming problems will be discussed in detail, along with a comparison of different discretization and solution methods.

Lastly, the thesis is concluded by an overview of the results, along with an outlook and possible areas for continued research in *Chapter 8*.

Chapter 2

Stochastic programs

General decision problems, when modelled in a mathematical way, consist of an objective function, decision variables, constraints and the problem data. The objective function usually maps from the set of decisions to a certain payoff, while the constraints can represent limits of resources or a certain pre-defined range for the decision variables. Mathematical programming thus deals with solving such instances by finding optimal decisions from the set of all available ones.

In many cases however, such problems can involve uncertainty. The aim is then to find an optimal decision with respect to this uncertainty, optimizing over multiple planning periods (e.g. multiple days). Being able to find such a solution assumes an either known or at least a reasonably well estimated probability distribution of the uncertain/random variables *and* the ability to evaluate the problem over the distributions (e.g. compute the corresponding integral).

The benefits of stochastic decision-making as opposed to a deterministic one are studied in great detail and have been proven to improve decisions in various areas, including Finance, Operations Research and Computer Science.

In the most known and well-studied case, stochastic programs consist of two stages (so-called two-stage-programs) and have a linear objective, along with linear equality (or inequality) constraints. Such problems consist of a first-stage decision, which is to be made before a random event occurs, and a recourse decision in the second stage, which can react to (and optimally improve upon) the outcome of the first-stage decision based on the now known realization of the uncertainty. The output of a two-stage stochastic program is thus a policy, consisting of a first-stage decision and an appropriate recourse decision which minimizes (maximizes) the expected future costs (profits). Such problems can be solved very efficiently in low dimensions by considering a so-called deterministic equivalent and solving it using standard LP-methods. A deterministic equivalent is essentially built by considering all outcomes of an either discrete or a discretized continuous random variable and optimizing with respect to every possible outcome simultaneously. However, as the number of possible outcomes grows, which can be especially problematic in a discretized continuous case, other solution methods become more relevant. The focus of efficient algorithms for solving two-stage SPs is to utilise the special problem structure in order to avoid solving the deterministic equivalent and still find an optimal (or approximately optimal) solution. One prominent approach utilising the problem structure

is the L-shaped method which is based on finding an outer linearisation of the recourse function and optimising with respect to a first-stage solution and an approximation of the recourse function. Many variations of this method are known, in particular regularisation and feasibility cuts.

A natural extension of the two-stage case is multi-stage stochastic programming. Multi-stage models allow for a sequence of decisions that react to the evolution of uncertainty over multiple periods. Such problems are in general more realistic and allow for more modelling flexibility. Many concepts and solution methods from the two-stage case can be transferred or generalised to the multi-stage case. However, this generalisation introduces various complications, especially due to the higher dimensionality of the problem. Additionally, the size of multi-stage programs (number of variables, number of constraints, number of nodes in a scenario tree) can grow exponentially fast with the number of random variables and/or the number of stages. It can be shown that in some cases, these problems can become PSPACE-hard. [14] Thus, efficiently solving multi-stage programs heavily relies on exploiting their specific structure in order to overcome the dimensionality problem. Such methods in general involve an approximation of the optimal solution and can be classified as follows [4]:

1. Value function approximations,
2. Constraint relaxation/dualization,
3. Policy restriction,
4. Time, state and path aggregation or scenario generation and reduction,
5. Monte Carlo methods.

For this thesis, case 1 and case 4 will be of particular importance.

In the remainder of this chapter, a formal introduction of stochastic programs is given along with some important notions, which will be used throughout this thesis.

2.1 Two-stage stochastic programs

A two-stage stochastic program is defined as follows:

Definition 2.1 (Two-stage stochastic program). *Let $\xi = (q, T, W, h)$ defined on $(\Omega, \mathcal{F}, \mathbb{P})$ and $Q(x, \xi(\omega)) = \min_x \{q^T y | Wy + Tx = h, y \geq 0\}$. Then the two-stage stochastic program has the form:*

$$\begin{aligned} \min \quad & c^T x + \mathbb{E}_\xi Q(x, \xi(\omega)) \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned} \tag{2.1}$$

The vector c represents the weights of first-stage decisions x . The vector ξ represents all random variables, whose future realization is unknown in stage 1. Further, x represents the first stage decision and y the recourse action, optimised with respect to a given first-stage decision. In addition, $Q(x) = \mathbb{E}_\xi Q(x, \xi(\omega))$ is called the recourse function of the problem.

As mentioned above, such problems can be solved via reformulation into its deterministic equivalent. This assumes that the underlying probability distribution of ξ is either discrete or can be approximated well enough via discretization. In both cases, ξ can be represented as a finite set of its possible outcomes (*scenarios*) $\xi(\omega_k)$ for all $\omega_k \in \Omega$. Each of these outcomes ω_k has a probability p_k with $\sum_k p_k = 1$.

Applied to this problem formulation, we define: $\xi_k = (q_k, T_k, W_k, h_k)$, each with probability p_k .

Definition 2.2. *The deterministic equivalent of the above problem is then defined as:*

$$\begin{aligned} \min_{x, y_1, \dots, y_k} \quad & c^T x + \sum_k p_k q_k^T y_k \\ \text{s.t.} \quad & Ax = b \\ & T_k x + W_k y_k = h_k, \forall k \\ & x, y_k \geq 0 \forall k \end{aligned} \tag{2.2}$$

As a linear problem, this deterministic equivalent formulation can in general be solved efficiently.

2.2 Multi-stage stochastic programs

Similar to the 2-stage case, a multi-stage program can be defined in the following way:

Definition 2.3. *Let $\mathcal{X}_t \subseteq \mathbb{R}^n$ be closed, convex feasible sets $\forall t \in [0, T]$, $\xi : (\Omega, \mathcal{F}, P) \rightarrow \mathbb{R}^m$ a random variable, $\xi = (\xi_1, \dots, \xi_T)$ with $\xi_t(\omega) \in \mathbb{R}^m$, $\xi^t = (\xi_1, \dots, \xi_t)$, $F : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, $F(x, \cdot)$ convex, measurable and integrable $F(\cdot, \xi) \forall x \in \mathcal{X}$, $x = (x_1, \dots, x_T)$ decisions with $x^t = (x_1, \dots, x_t)$ and $x_t \in \mathcal{X}_t$. Then, the general form of a multistage problem is given by:*

$$\max_{x \in \mathcal{X}} \mathbb{E}_P [F(x, \xi)] \tag{2.3}$$

An alternative form can be found by defining value functions:

$$\begin{aligned} V_t(x^{t-1}, \xi^t) &= \min_{x_t \in \mathcal{X}_t} f_t(x^t, \xi^t) + \mathbb{E}[V_{t+1}(x^t, \xi^{t+1}) | \xi^t], \quad \forall t : 1 \leq t < T \\ V_T(x^{T-1}, \xi^T) &= \min_{x_T \in \mathcal{X}_T} f_T(x^T, \xi^T) \end{aligned} \quad (2.4)$$

Then, the objective value of the multi-stage problem becomes

$$\min_{x_1 \in \mathcal{X}_1} f_1(x_1, \xi_1) + \mathbb{E}[V_2(x^1, \xi^2) | \xi^1] \quad (2.5)$$

Which is equivalent to the nested representation:

$$\min_{x_1 \in \mathcal{X}_1} f_1(x_1, \xi_1) + \mathbb{E} \left[\min_{x_2 \in \mathcal{X}_2} f_2(x_2, \xi_2) + \mathbb{E} \left[\dots + \mathbb{E} \left[\min_{x_T \in \mathcal{X}_T} f_T(x_T, \xi_T) \right] \right] \dots \right] \quad (2.6)$$

Note that since a closed form expression of $x \mapsto \mathbb{E}_P[F(x, \xi)]$ is rarely available, integrals $\mathbb{E}[V_t(x^{t-1}, \xi^t) | \xi^{t-1}]$ need to be calculated using a discretization of ξ . This will be discussed in more detail in Chapter 3.2.

Further important notions needed later on:

Definition 2.4 (Resource state). *For a given stochastic program, the resource state $R_t = R_t(x_1, \xi_1, \dots, x_{t-1}, \xi_{t-1})$ is defined as the set of all variables describing the available resources at a given stage of a stochastic optimization problem. This definition can be further extended by the so called information state I_t and knowledge state K_t as described in [42].*

Definition 2.5 (Policy). *For a given stochastic optimization problem, a policy $\pi(R_t)$ is a decision function, mapping from the resource state to a feasible action.*

*A policy is called **admissible** if it does not depend on the future, i.e. a policy which is completely defined by the information known until time t . An **implementable** policy describes a policy with feasible decisions. [42]*

Definition 2.6 (Relatively complete recourse). *A given multi-stage SP, is said to have relatively complete recourse, if $\forall (x_1, \dots, x_t), (\xi_1, \dots, \xi_t) \exists (x_{t+1} \dots x_T)$ with $(x_{t+1} \dots x_T)$ being feasible with probability 1. The problem is said to have **complete recourse** if this is the case for any sequence of decisions and random variables up to time t , i.e. the feasibility of the later stages is independent of the decisions in the earlier stages.*

2.3 Measures for evaluating solutions

In the subsequent chapters, different approaches to solving stochastic programs will be compared to each other. While we introduce a new approach suited for comparing multi-stage programs in *Chapter 4*, there are also well-known measures allowing to assess the quality of a given stochastic solution.

First measure of this kind is called EVPI (expected value of perfect information). EVPI can loosely be defined as the amount a decision-maker would be ready to pay in order to know the future outcomes perfectly (i.e. completely and accurately). For a formal definition, we first need to define the notion of a *wait-and-see-solution* and of a *here-and-now-solution*.

Definition 2.7 (Wait-and-see-solution). *Let the solution of a two-stage problem as defined in 2.1 for one particular realization of ξ be*

$$\min z(x, \xi) = c^T x + \min\{q^T y | Wy = h - Tx, y \geq 0\}, \text{ s.t. } Ax \leq b, x \geq 0. \quad (2.7)$$

Further assume that such a z always exists (otherwise the SP is not well-defined), i.e. $\forall \xi(\omega) \exists x \in \mathbb{R}$ s.t. $z(x, \xi) < \infty$. Further, define $x^(\xi)$ as an optimal solution of Problem 2.7. Assume we can always find $x^*(\xi)$ and the corresponding objective value $z(x^*(\xi), \xi)$. Then the wait-and-see solution is given by:*

$$WS = \mathbb{E}_\xi [z(x^*(\xi), \xi)] \quad (2.8)$$

Definition 2.8 (Here-and-now-solution). *The here-and-now-solution corresponding to the recourse problem (RP) is defined as:*

$$RP = \min_x \mathbb{E}_\xi [z(x, \xi)] \quad (2.9)$$

Definition 2.9 (Expected value solution). *Let z be defined as in Problem 2.7. Then the expected value solution is defined as: $EV = \min_x z(x, \bar{\xi})$, where $\bar{\xi} = \mathbb{E}[\xi]$ and the expected result of using this solution (EEV) can be found by:*

$$EEV = \mathbb{E}_\xi [z(x^*(\bar{\xi}), \xi)] \quad (2.10)$$

Note that EEV can be thought of as measuring the performance of an "averagers solution". Also, in general, this solution does not have to be optimal in any way, especially given that non-optimal EEVs are the reason for calculating a stochastic solution in the first place.

Having introduced the fundamentals, we can turn towards two important measures, EVPI and VSS which can be defined as follows:

Definition 2.10 (EVPI = expected value of perfect information).

$$EVPI = RP - WS \quad (2.11)$$

and

Definition 2.11 (VSS = value of stochastic solution).

$$VSS = EEV - RP \quad (2.12)$$

In the following, it will be useful to know the following relationship between these two measures:

Proposition 2.12. *For all stochastic programs,*

$$0 \leq EVPI \leq EEV - EV \quad (2.13)$$

and

$$0 \leq VSS \leq EEV - EV \quad (2.14)$$

Proof. See [4]. □

Since both EEV and EV can be computed with reasonable effort, the difference represents an easy way to check for an upper boundary of a stochastic solution. This relationship is often depicted as shown in *Figure 2.1*.

Further research on the relationship between these two values can be found in [4].

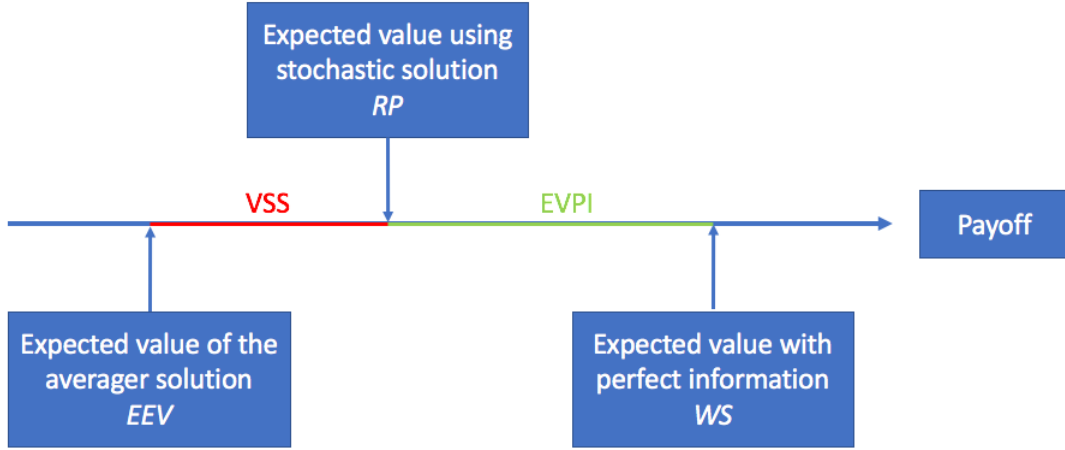


Figure 2.1: Overview of solution measures

2.4 Markov Decision Process (MDP)

Let us now introduce the notion of a Markov Decision Process, which we will build upon in later chapters.

Definition 2.13 (Markov Decision Process). *Assume that the set of resources R_t is given at time t . A realization of a state is denoted by r_t . Further assume there is a (finite) set of possible decisions x_t at every state t . The transition probabilities between states have to fulfil the Markov property, i.e. $\mathbb{P}(R_{t+1} = r_{t+1} | r_t, x_t) = \mathbb{P}(R_{t+1} = r_{t+1} | r_0, \dots, r_t, x_0, \dots, x_t)$. Moreover, the Markov property can be expressed using value functions in the following way: If ξ is Markov and the value function V_t only depends on the current state, i.e.,*

$$\mathbb{E}[V_t(x^{t-1}, \xi^t) | \xi^{t-1}] = \mathbb{E}[V_t(R_t, \xi_t) | \xi_{t-1}],$$

then the problem is called a Markov Decision Process.

Definition 2.14 (Environmental state). *While the resource state consists of variables, which can be influenced by the decisions x_t , the environmental state cannot be influenced and describes the evolution of the variables not influenced by the decision-maker, e.g. the price or the demand for a certain good.*

Proposition 2.15. *If the state variable is complete, every dynamic program is Markovian.*

Proof. See [42]. □

Solving MDPs involves approaches which rely on full or partial discretization (discrete event and action space). The known algorithms are Dynamic Programming, Value Iteration and Policy Iteration algorithms. For an overview of these methods, consider:

- *L-Shaped Method:* [52]
- *Benders decomposition:* [3], [38]
- *Stochastic Dual Dynamic Programming:* [40]

- *Approximate Dual Dynamic Programming* (ADDP): [35, 36]

And a survey: [9]. In *Chapter 3.5* an approach using a discretization of the environmental state as a lattice, along with a continuous action and resource state will be introduced.

Chapter 3

Solution methods

As mentioned in *Chapter 2*, various solution methods can be employed to solve multi-stage stochastic programs by either exploiting their structure and/or finding a discretization of the underlying random variables. In this chapter, along with some basic notions, two different approaches to solving multi-stage SPs will be introduced. The first approach relies on approximating the distribution of random variables via a scenario tree and using it to evaluate the value functions (as defined in *Chapter 2.2*) by formulating a deterministic equivalent of the problem over all possible scenarios. The second approach is designed specifically for Markov Decision Processes (as defined in *Chapter 2.4*) and employs a scenario lattice to discretize the event space.

3.1 Curse of dimensionality

As a preliminary to the following discussion, we want to shortly argue why the well-known curse of dimensionality is relevant in stochastic optimization, especially in the multi-stage case. For a more detailed survey, refer to [41]. Since stochastic optimization heavily relies on being able to approximate a continuous distribution by sampling, there are in general two drivers contributing to the curse of dimensionality: the number of random variables considered per stage and the number of stages. Both are increasing the total number of problem variables, since every SP has a total of $|S| \times |x| \times |T|$ variables, where S is the set of scenarios and T is the number of stages. As a result, stochastic optimization problems scale non-polynomially with the increase of the underlying dimension. Apart from the necessity of solving integrals in high dimensions, multi-stage problems additionally introduce the need of finding a discretization of conditional distributions.

Apart from the scaling issues, being able to successfully sample from a multivariate probability distribution presents a problem, since the number of points required to "cover" a higher-dimensional space grows exponentially in the number of dimensions. This can be understood easily by considering *Figure 3.1*. With growing number of dimensions, the volume of the unit hypersphere becomes increasingly insignificant as compared to the volume of the unit hypercube. This, in turn, means that the points in the hypercube end up being further and further "away" from the origin. In order to still cover the space evenly, more points need to be sampled. For example, 100 evenly-spaced sample points suffice to sample a unit interval with no more than 0.01 units distance between points.

An equivalent sampling of a 10-dimensional unit hypercube with a grid with spacings of 0.01 between adjacent points would require 10^{20} sample points: thus, in some sense, the 10D hypercube can be said to be a factor of 10^{18} larger than the unit interval. See [31]. Various techniques for efficient high-dimensional sampling have been proposed, such as Importance Sampling, Stratified Sampling and Latin Hypercube Sampling. For further references on the curse of dimensionality, see [15],[45],[47],[49].

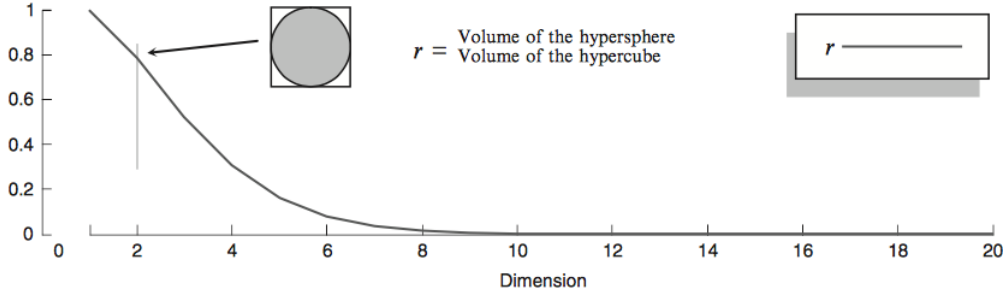


Figure 3.1: Ratio volume(hypersphere)/volume(hypercube), see [31]. As we can see, the ratio goes to zero for growing Dimension.

3.2 Scenario trees and their construction

We begin with the theory behind scenario trees in stochastic optimization based on the survey presented in [12].

Assumption 3.1. *The probability distribution $\mathbb{P}$ of ξ is discrete and has finite support.*

Definition 3.1 (Scenario). *A sequence of realizations $\xi^t = \{\xi_1, \dots, \xi_t\}$ is called scenario at stage t . The set of all scenarios can be described by $\mathcal{S} = \{\xi^1, \dots, \xi^S\} = \{\xi | \xi_t \in \mathcal{S}_t(\xi^{t-1}) \forall t > 1\}$, where $\mathcal{S}_t(\xi^t)$ is the support of the conditional probability distributions of ξ_t conditioned on all prior realizations.*

Definition 3.2 (Arc probabilities, Path probabilities). *Arc probabilities are the conditional probabilities of ξ_t conditioned on ξ^{t-1} , i.e. $\mathbb{P}(\xi_t | \xi^{t-1})$. Path probabilities are defined as the products of arc probabilities up to time $t - 1$, i.e. $\mathbb{P}(\xi^{t-1}) = \mathbb{P}(\xi_1) \prod_{i=2}^{t-1} \mathbb{P}(\xi_i | \xi^{i-1})$.*

In order to model the data behind a stochastic optimization problem in a way consistent with the definitions above, a scenario tree can be used.

Definition 3.3 (Scenario tree). *A scenario tree is an oriented graph consisting of edged and nodes, where every node has a unique predecessor. Each node represents a possible state and each edge represents a transition between states. The set of nodes V can be disjointly partitioned into $V = \dot{\cup} \mathcal{N}_t$ with $\mathcal{N}_t$ being the nodes belonging to stage t . Moreover, a probability p_i is assigned to every edge $e_i \in E$, representing the arc probability as defined above.*

An example of a scenario tree is presented in Figure 3.2a. An important special case of a tree is called a fan.

Definition 3.4 (Scenario fan). *A scenario fan is a special case of a scenario tree, where each scenario $\xi^s = \{\xi_1^s, \dots, \xi_T^s\}$ has probability $p_s = \mathbb{P}(\xi_1^s)$ and is independent of other scenarios. A fan is usually constructed directly from simulated path of a stochastic process where every path is equally probable and independent of others.*

An example of a scenario fan is presented in Figure 3.2b. The conditional probability of a certain scenario occurring given that one of the successors of the root node occurred is 1, i.e. once a path is chosen, the remaining steps are deterministic. For this reason, a fan can be reduced to a 2-stage stochastic program.

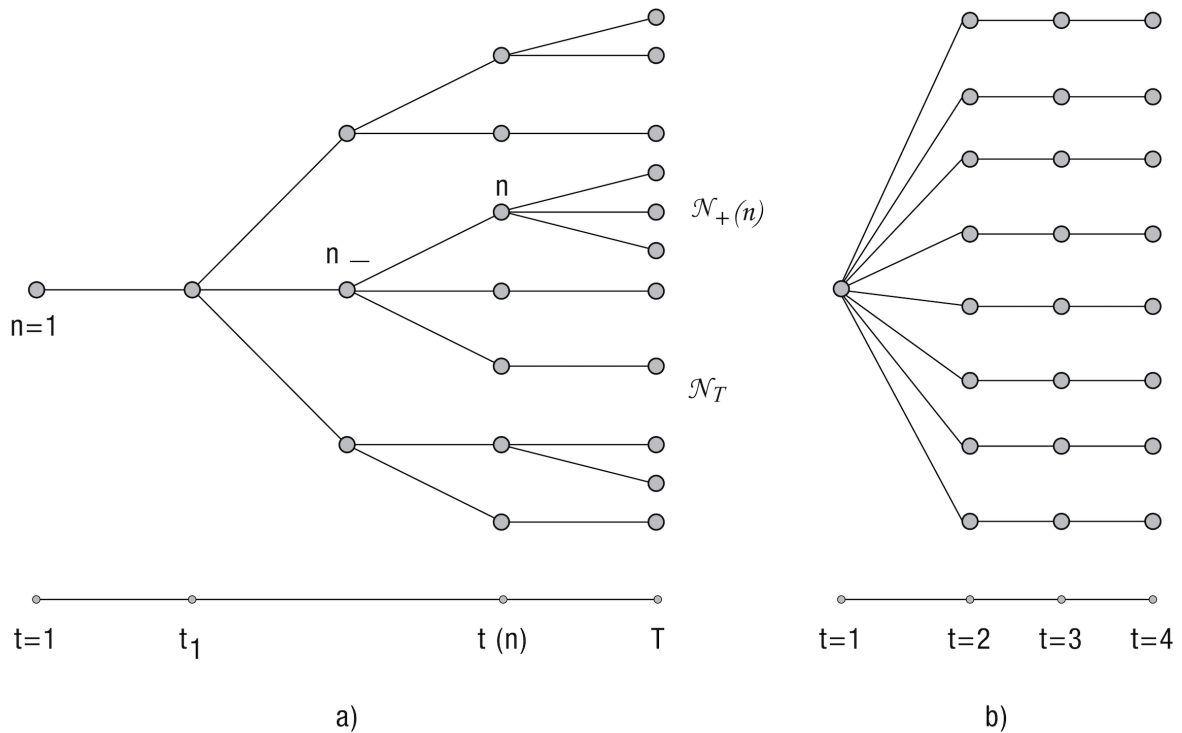


Figure 3.2: An exemplary scenario tree (a) modelling 11 scenarios and an exemplary scenario fan (b) modelling 8 scenarios.

Constructing scenario trees is a well-researched topic with different approaches discussed in the literature. We will in the following give a short overview of commonly applied approaches, along with their advantages and drawbacks. For further surveys, we refer to [30]. It should be noted that the methods for constructing scenario trees strongly depend on the structure of the given decision problem, as well as the underlying stochastic processes. Therefore, there is no unique established way to construct scenario trees. Rather, there are different approaches depending on the structure of the optimization problem. Apart from this difficulty, the construction of suitable scenario trees presents a trade-off between computational tractability and exactness of solutions. There is also a gap between computationally effective methods and theoretically well-founded ones. In many applications, the construction of a scenario tree is tightly connected to the modelling of the stochastic process. While some methods combine sampling and construction, other

methods assume that the sample paths of the random processes have already been generated or can be generated according to a certain type of model (e.g. a time-series model). Constructing a scenario tree also requires a decision to be made about the number of stages and branching. Both are strongly linked to the problem domain. It should be noted that the length of stages in a SP is not necessarily uniform, especially in models where the end-of-horizon effects are included (i.e. what happens after the last stage of the planning).

When modelling scenario trees for financial applications, it is also important to make sure that no implicit arbitrage opportunities are possible. This requirement will be particularly important in *chapter 7*.

Overview of scenario tree construction approaches

Internal sampling and decompositions

In case of internal sampling, possible scenarios are being constructed while solving the decision problem. In this case, no explicit ex-ante scenario tree is required. The information on how to sample is acquired by e.g. considering the dual variables. An advantage is that the scenario tree does not need to be constructed explicitly. A disadvantage of this approach is that the model is required to be linear. Examples of internal sampling and decomposition-based approaches can be found in [25] and [27] (combined with Benders decomposition). The decision on how many stages to use still has to be made.

Bootstrapping

Bootstrapping is a non-parametric approach which does not rely on modelling of the underlying stochastic process. Instead, known and recorded historical realizations are being bootstrapped to generate scenarios. This approach is advantageous when no sensible assumption about the form of the stochastic process can be made. The drawback is a potential bias due to unobserved realizations. Another drawback is that all scenarios are being weighted equally which does not necessarily reflect their true probabilities of occurring. For further considerations, see [53].

Conditional sampling

The basic idea of conditional sampling is to traverse over a tree and sample a number of values from the underlying stochastic process at every node to create scenarios rooted in that particular node. In order to apply conditional sampling, the conditional distribution of the stochastic process has to be known for every node. Constructing trees this way leads to an exponential increase in the total number of nodes. This sampling technique becomes especially difficult, if multidimensional correlated random variables are required [30]. Principal component analysis can be used in combination with this method in order to reduce the number of dimensions and dampen the problem of having to generate correlated random vectors, as shown in [37], [50], [39], [28]. The method of conditional sampling is closely related to sampling from marginals combined with a correlation matrix, as seen for example in [8].

Moment matching

The idea behind moment matching is to generate (possibly multidimensional) random variables with the first four marginal moments corresponding to the theoretical model (or fitted data) and with a predefined correlation structure. This requires an iterative procedure during which the random variables are being generated according to their moments, then transformed to meet the correlation structure. This transformation, however, shifts the moments, which need to be readjusted. For a more detailed description, see [26]. One of the advantages of moment matching is that in a multidimensional setting, generating a high number of scenarios is computationally tractable and fairly easy, even for heavy-tailed distributions. One of the disadvantages is that there is no general proof of convergence, i.e. there is no link between a solution obtained over a moment-matched scenario tree and the theoretical solution which could have been obtained by optimizing over the true distribution. While moment matching has seen a wide use in practice, the constructed trees need to be checked ex-post for desired properties and the number of nodes can rise exponentially. Apart from that, there is no guarantee that a sensible scenario tree is generated, especially in settings where the predefined moments and correlation structure contradict each other.

Path-based methods / Clustering

When scenario generation is decoupled from tree construction, path-based methods can be applied. These rely on clustering where the random vector ξ is clustered according to ξ_1 first. Afterwards, conditional clustering with respect to $\xi_2, \xi_3 \dots$ is performed. As described in [12], clustering can take place according to a suitable measure of similarity between scenarios, such as for example

$$d_1(\xi^i, \xi^j) = \frac{1}{T} \sum_{t=1}^T w_t |\xi_t^i - \xi_t^j|$$

$$d_2(\xi^i, \xi^j) = \frac{1}{T} \sum_{t=1}^T w_t (\xi_t^i - \xi_t^j)^2$$

where w_t represent weights, chosen such that the emphasis is put on certain parts of the whole scenario (or uniformly). These measures can subsequently be used to create clusters. At first, clusters $C_1^1, \dots, C_1^n$ are created based on the first component. Then, for each cluster C_1^i , the same procedure is repeated based on the second component. This is continued iteratively for the length of a scenario. An advantage of this method is the independence between scenario generation and tree construction. A drawback is once again computational intractability and the fact that the number of nodes grows exponentially with the number of stages *and* the number of clusters per stage.

Optimal approximations w.r.t. probability metrics

This approach requires a predefined probability metric, according to which the construction takes place. One such approach as introduced in [24]:

$$d_{\mathcal{F}}(P, Q) = \sup_{f \in \mathcal{F}} \left| \int_{\Xi} f(\xi) P(d\xi) - \int_{\Xi} f(\xi) Q(d\xi) \right| \quad (3.1)$$

and in the linear case:

$$\mathcal{F}_c = \{f : \Xi \rightarrow \mathbb{R} | f(\xi) - f(\tilde{\xi}) \leq c(\xi, \tilde{\xi}), \forall \xi, \tilde{\xi} \in \Xi\} \quad (3.2)$$

and a cost function

$$c(\xi, \tilde{\xi}) = \max\{1, \|\xi - \xi_0\|^{r-1}, \|\tilde{\xi} - \xi_0\|^{r-1}\} \|\xi - \tilde{\xi}\| \quad (3.3)$$

Such metrics are called Forter-Mourier metrics of order r . The scenario construction can also be done with respect to the Monge-Kantorovich functional, defined as ([23]):

$$\mu_c(P, Q) = \inf \left\{ \int_{\Xi \times \Xi} c(\xi, \tilde{\xi}) \eta(d\xi, d\tilde{\xi}) : \eta \in M(P, Q) \right\} \quad (3.4)$$

where $c()$ is again a cost function.

Now, assuming P is the probability distribution of $\xi_t, \forall t \in [1, T]$, the aim is to select $c()$ such that the underlying SP is stable with respect to μ_c , as described in [23]. Given $P, \epsilon > 0$, we are looking for a tree with the distribution P_{tr} such that

$$\mu_c(P, P_{tr}) \leq \epsilon \quad (3.5)$$

The construction of the tree takes place based on a scenario fan of simulated scenarios, together with stage-by-stage approximations. This is visualized in *Figure 3.3*.

Connected to the notion of scenario tree construction is the reduction of scenario trees. This becomes particularly relevant, when the size of the problem makes it computationally intractable with current algorithms. The idea behind scenario reduction is to summarize distinct scenarios in a single scenario and thus reduce the total number of scenarios, without losing "too much" of the probability information, with respect to a probability metric as defined above. Again, a certain ϵ percentage precision is defined and scenario reduction can take place via either forward selection or backward reduction.

3.3 Solutions based on scenario trees

Assume we have a scenario tree built according to one of the methods described in the previous section. The next step is to solve the SP by formulating its deterministic equivalent over the constructed tree. This can be achieved by assigning a decision variable x_n to every node of the tree, $n \in \mathcal{N}_t, \forall t \in [0, T]$. Using these decision variables and the deterministic equivalent, we are now in position to use a solver to find the optimal decisions $(x_n)^*, \forall n$, including the first-stage-decision x_1 .

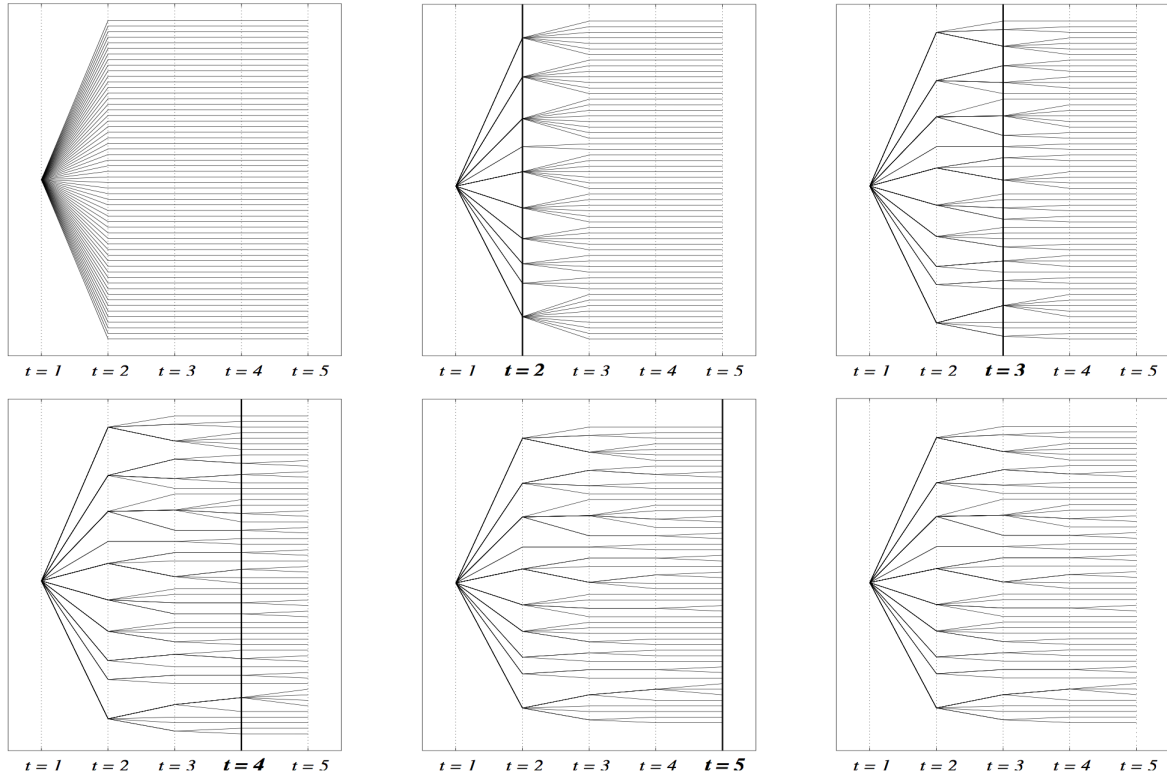


Figure 3.3: An example of a scenario tree construction by forward reduction. Source: [22]

3.4 MDPs and scenario lattices

As demonstrated in the previous chapter, scenario trees are a well-researched topic in stochastic optimization. However, regardless of the many possible ways to construct a scenario tree, the issue of dimensionality and of exponential tree size growth is unavoidable. These problems are largely due to the restriction that every node is required to have a unique predecessor. While this restriction allows for a unique mapping from an ending node back to the whole history of the process, it is not always necessary to consider the complete history of a scenario. However, when modelling SPs as Markov Decision Problems (see *Chapter 2.4*), we can relax this restriction and consider *scenario lattices* instead. The difference to scenario trees is that in a lattice, each node can have multiple predecessors. An example of a scenario tree and a comparable scenario lattice can be seen in *Figure 3.4*. Apart from the obvious advantage of modelling more scenarios by the same number of nodes, using the combination of scenario lattices and MDP also allows us to split the state space up into an environmental and a resource state. This will become important in the next chapter where we consider an algorithm for solving MDPs over lattices.

As can be seen in the *Figure 3.4*, an exemplary scenario lattice with half the number of nodes can be used to model more than 7 times the number of scenarios of an exemplary tree. This is due to the fact that a tree can represent at most $|\mathcal{N}_T|$ scenarios while a scenario lattice represents up to $|\mathcal{N}_1| \times |\mathcal{N}_2| \times \dots \times |\mathcal{N}_T|$ scenarios.

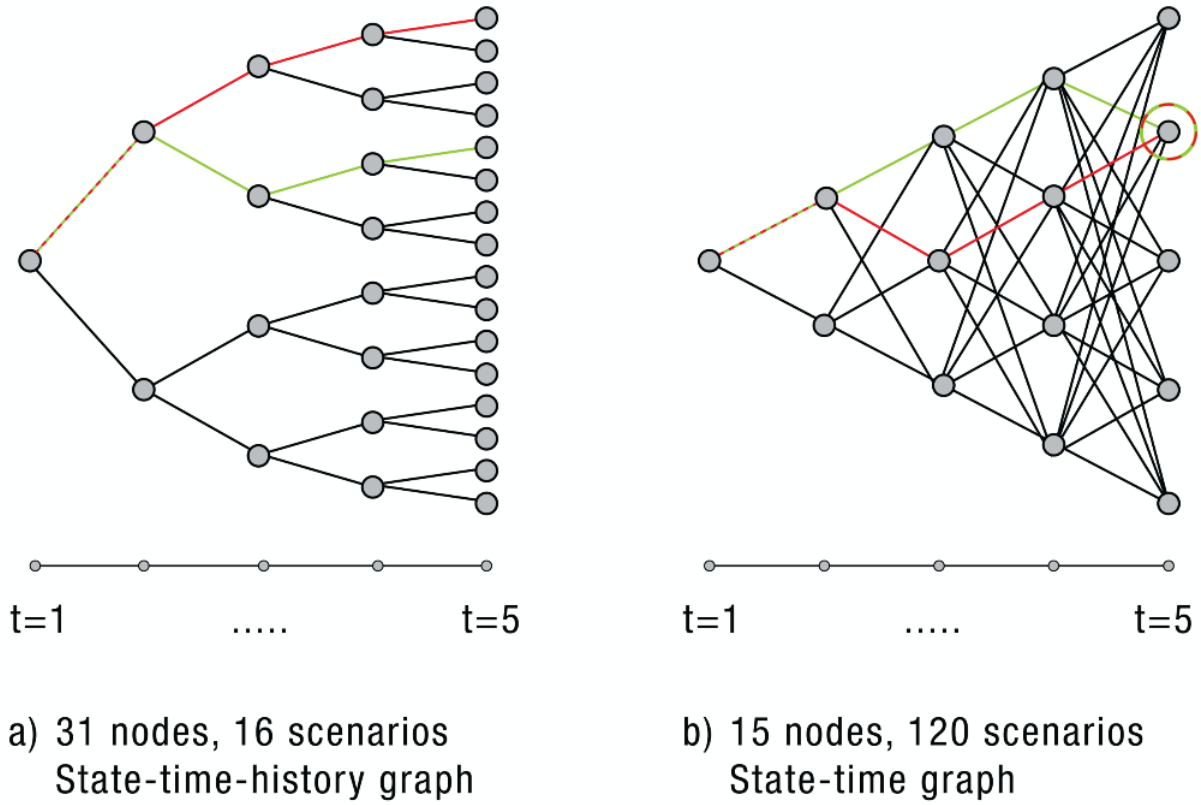


Figure 3.4: An exemplary scenario tree (a) and an exemplary scenario lattice (b)

As noted above, a scenario lattice does not allow to trace back the history of a given scenario based on its ending node. This can be easily seen from *Figure 3.4b*, where the marked ending node can have multiple paths leading to it.

3.5 Solving MDPs and approximate dual dynamic programming

Using the framework presented above, we now want to solve a given Markov Decision Problem using a scenario lattice. In this chapter, we follow the approach presented in [35],[36].

As described earlier, solving an MDP is equivalent to finding the functions $S \mapsto V_t(S, \xi_t)$. We now want to utilize the fact that the state space of an MDP can be separated into resource state and environmental state.

In order to be able to use Approximate Dual Dynamic Programming to solve a given Markov Decision Problem, a few assumptions need to be made:

- We have a linear / convex optimization problem,

- the environmental state can be modelled as a Markov process and discretized using a scenario-lattice,
- the problem has relatively complete recourse as defined in *Definition 2.6*,
- the resource state only appears on the RHS of constraints.

In the next step, in order to find the value functions, observe that for each realization $\hat{\xi}_T$ of the discretized Markov process, the function $S \mapsto V_T(S, \xi_T)$ can be approximated by a minimum of its supergradients. This is due to the fact that the map is a piecewise linear, concave function (Proof: [36]). In order to find states S at which to calculate the supergradients, we can use the current approximation of the value function. This procedure can be summarized in the following algorithm:

- Input: $S \mapsto \hat{V}_t(R, \hat{\xi}_t)$, $\forall t$, $\forall \hat{\xi}_t$ (current approximation of the value function, realization of the random variable).
- *Forward-Pass*
 - Sample path $(\hat{\xi}_1, \dots, \hat{\xi}_T)$ from lattice
 - Decide on x_t using $\hat{V}_t(R, \hat{\xi}_t)$ and store resource path $(r_1, \dots, r_T)$
- *Backward-Pass* (from $t = T, \dots, 1$)
 - At t calculate tangent l to $R \mapsto \hat{V}_t(R, \hat{\xi}_t)$ at point r_t

$$\hat{V}_t(r_t, \hat{\xi}_t) = \begin{cases} \min_{x_t} & c_{\hat{\xi}_t}^\top x_t + \mathbb{E} [V_{t+1}(R_{t+1}, \hat{\xi}_{t+1}) | \hat{\xi}_t] \\ \text{s. t.} & W_{\hat{\xi}_t} x_t \leq b_{\hat{\xi}_t} + B_{\hat{\xi}_t} S_t \end{cases}$$
 - $\hat{V}_t(R, \hat{\xi}_t) \leftarrow \min(l, \hat{V}_t(R, \hat{\xi}_t))$ (updating the current approximation).

Proposition 3.5 (Value function approximations). *The value function approximations in any iteration are upper bounds for the real value functions. Proof: [35].*

Proposition 3.6 (Convergence). *The policies obtained from single cut ADDP converge to the optimal policies in finitely many iterations. Proof: [35].*

In order to define a stopping criterion for the ADDP algorithm, we need to define upper and lower bounds and check the gap between them. As soon as the gap falls below a predefined threshold, the algorithm should terminate.

Definition 3.7 (Lattice upper bound *LUB*). *The upper bound of the objective value on the lattice, *LUB*, is given by:*

$$LUB = V_1(P_0, R_1) = \max_{\pi_1 \in \Pi(R_1)} \{C(P_0, \pi_1, R_1) + \hat{V}_2(P_0, R_2)\}. \quad (3.6)$$

where $C(.,.)$ denotes the reward function and $\hat{V}$ is the current approximation of the value function and π denotes a policy.

Definition 3.8 (Lattice lower bound LLB). *Lattice lower bound LLB is defined as:*

$$LLB = \frac{1}{N} \sum_{i=1}^N \Pi_n \quad (3.7)$$

where Π denotes the resulting revenue generated in each forward pass.

Observe, that LLB is a random variable. The convergence gap can be defined as the difference $LUB - LLB$. Alternatively, the total amount of iterations done by ADDP can be fixed to a certain number, assuming that the approximation after this total amount of iterations is sufficiently good.

Chapter 4

Analyzing and comparing solutions

Having established the two approaches to solving multistage SPs which are of interest for us, the next interesting question is to compare them and decide which one performs better in a given case. Thus, the aim of this chapter is to discuss how the tree-based deterministic equivalent solutions can be compared to those found by using ADDP. As a prerequisite for such a comparison, we first identify 6 criteria for differentiating between problem classes. After that, we present standard models discussed in the stochastic optimization literature which differ with respect to the presented criteria. In the next step, we introduce measures which can be used for such comparisons and argue why the proposed comparison setup allows for a fair and precise comparison of these two methods.

4.1 Problem classes

For the comparison case studies, we identified the most important properties of multistage stochastic programs. Thus, multistage SPs can be differentiated by the following properties:

- (i) Randomness in the objective
- (ii) Randomness in the constraints
- (iii) Varying dimensionality: varying number of random variables involved
- (iv) Varying dimensionality: varying planning horizon
- (v) Risk-averse (CVaR) formulation of the objective
- (vi) Underlying stochastic process to model the random variables

The first two properties are fairly straightforward: the random variable in a given SP can either be a part of the objective function or a part of the constraint. For some models, it can be found in both, objective and constraint. The aggregate production planning (APP) model and the asset liability management model (ALM) presented in *chapter 5* and *chapter 7* contain a random variable "demand" respective "liability", which can be found in the constraints only. The classic flower girl problem presented in *chapter 6* contains randomness in both, objective and constraints. Randomness in constraints is due to

random demand which is to be satisfied by the flower girl and randomness in the objective is due to the random price of ordering flowers.

The next two criteria are representing the dimensionality of a SP: There are in general two ways to change the dimensionality (iii) and (iv): either the number of random variables per stage or the number of stages can be increased. Both leads to a higher dimension of the stochastic program. This, in turn, can cause the curse of dimensionality, as described in *section 3.1*.

Criterion (v) is differentiating between risk-neutral and risk-averse formulation of the policy of a SP. This criterion is explored by a risk-averse variation of the classic flower girl problem in *chapter 6*. This criterion is particularly interesting, because the CVaR objective formulation in connection with stochastic programming is causing various additional technical difficulties, with time consistency being one of the important ones. [46]

Lastly, the kind of underlying stochastic process (vi) also can be varied to include many different kinds of processes. An overview of the processes included in the case studies is the subject of the next section.

4.2 Stochastic processes

In this section, we want to introduce various stochastic processes which will later be required for modelling the random variables in SPs. All of the introduced processes are widely used in practice.

Definition 4.1 (Lévy process). *A real-valued, cadlag, stochastic process L_t with $L_0 = 0$ is called Lévy process, if for any $s, t \in [0, T]$ with $s < t$,*

- (i) *L has independent increments, i.e. $L_t - L_s$ is independent of $\mathcal{F}_s$.*
- (ii) *L has stationary increments, i.e. the distribution of $L_{t+s} - L_t$ is independent of t .*
- (iii) *L is stochastically continuous, i.e. $\lim_{s \rightarrow t} P(|L_t - L_s| > \epsilon) = 0$*

Geometric Brownian Motion

Definition 4.2 (Brownian Motion). *A standard Brownian Motion is a random process $W = \{W_t : t \in [0, \infty)\}$ with state space $\mathbb{R}$ and the following properties:*

- (i) *$W_0 = 0$ a.s.*
- (ii) *W has stationary increments: $\forall s, t \in (0, \infty)$, the distribution of $W_t - W_s$ is the same as the distribution of W_{t-s}*
- (iii) *Independent increments: for $t_1, t_2, \dots, t_n \in (0, \infty)$, $t_1 < t_2 < \dots < t_n$, $W_{t_1}, W_{t_2} - W_{t_1}, \dots, W_{t_n} - W_{t_{n-1}}$ are independent.*
- (iv) *$W_t \sim N(0, t)$ for each $t \in [0, \infty)$*
- (v) *The map $t \mapsto W_t$ is continuous on $[0, \infty)$.*

Definition 4.3 (Geometric Brownian Motion). *Let $W = \{W_t : t \in [0, \infty)\}$, $\mu \in \mathbb{R}, \sigma \in (0, \infty)$. Then*

$$X_t = X_0 \exp\left(\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma W_t\right), t \in [0, \infty) \quad (4.1)$$

is called geometric Brownian motion with W Brownian Motion, drift parameter μ , volatility parameter σ and starting value X_0 .

Also note that the above process can be seen as the solution of the following stochastic differential equation:

$$dX_t = \mu X_t dt + \sigma X_t dW_t \quad (4.2)$$

Geometric Brownian Motion can be used to e.g. model the evolution of stock prices, population growth and many other random processes with noise. Note that as compared to the Brownian Motion, GBM assumes positive values only. For a reference, see [1]. To use GBMs within the context of this thesis, we will need some more definitions:

Proposition 4.4. *For $t \in (0, \infty)$, X_t is log-normally distributed with parameters $(\mu - \frac{\sigma^2}{2})t$ and $\sigma\sqrt{t}$. The corresponding density f_t is given by:*

$$f_t(x) = \frac{1}{\sqrt{(2\pi t)\sigma x}} \exp\left(-\frac{[\ln(x) - (\mu - \frac{\sigma^2}{2})t]^2}{2\sigma^2 t}\right), x \in (0, \infty) \quad (4.3)$$

Proof. See [44]. □

Proposition 4.5 (Moments). *For $n \in \mathbb{N}$ and $t \in [0, \infty)$,*

$$\mathbb{E}(X_t^n) = X_0^n \exp(n\mu t) \quad (4.4)$$

In particular,

$$\mathbb{E}(X_t) = X_0 \exp(\mu t) \quad (4.5)$$

$$\text{Var}(X_t) = X_0^2 \exp(2\mu t)(\exp(\sigma^2 t) - 1) \quad (4.6)$$

Proposition 4.6 (Martingale Property). *If $\mu = 0$, GBM is a martingale with respect to the Brownian motion W .*

Proof. Assume $\mathcal{F} = \{\mathcal{F}_t : t \in [0, \infty)\}$ with $\mathcal{F}_t = \sigma\{W_s : 0 \leq s \leq t\}$ is the natural filtration of the Brownian Motion. Let $s, t \in [0, \infty)$ with $s \leq t$. Observe that $W_t = W_s + (W_t - W_s)$. We can now reformulate GBM as:

$$X_t = \exp\left(-\frac{\sigma^2}{2}t + \sigma W_s + \sigma(W_t - W_s)\right) \quad (4.7)$$

Now,

$$\mathbb{E}(X_t | \mathcal{F}_s) = \exp\left(-\frac{\sigma^2}{2}t + \sigma W_s\right) \mathbb{E}(\exp(\sigma(W_t - W_s)))$$

and by substituting the normal distribution of the increment $W_t - W_s$, we get:

$$\mathbb{E}(X_t | \mathcal{F}_s) = \exp\left(-\frac{\sigma^2}{2}t + \sigma W_s\right) \exp\left(\frac{\sigma^2}{2}(t - s)\right) = X_s$$

□

For the purposes of simulating correlated random variables, we will also require a multivariate version of the GBM.

Definition 4.7 (Multivariate GBM). *In the multivariate version, the SDE from Equation 4.2 changes to:*

$$dX_t^i = \mu_i X_t^i dt + \sigma_i X_t^i dW_t^i \quad (4.8)$$

and the BMs W_i can be correlated, i.e. $\mathbb{E}(dW_t^i, dW_t^j) = \rho_{i,j} dt$. The corresponding covariance is:

$$\text{Cov}(X_t^i, X_t^j) = X_0^i X_0^j \exp((\mu_i + \mu_j)t) (\exp(\rho_{i,j} \sigma_i \sigma_j t) - 1) \quad (4.9)$$

see [44].

Stochastic jump processes

When modelling random price processes, it is often allowed for the price to jump at a random point of time by a random amount. This behaviour can be modelled by various jump processes.

Definition 4.8 (Counting process). *A counting process is a stochastic process $N_t, t \in \mathbb{R}_+$ with non-negative, integer, non-decreasing values, i.e.*

- (i) $N(t) \geq 0$
- (ii) $N(t)$ integer
- (iii) For $s \leq t$, $N(s) \leq N(t)$

Definition 4.9 (Poisson process). *Poisson process is a stochastic counting process $N_t, t \in \mathbb{R}_+$ which fulfils the following conditions:*

- (i) $N(0) = 0$
- (ii) $N(t)$ has independent increments
- (iii) $N(t) - N(s) \sim \text{Poi}(\lambda(t - s))$ for $s < t$.

Where Poi is the Poisson distribution density with intensity λ , i.e. for $X \sim \text{Poi}$, $\mathbb{P}(X = k) = \frac{\lambda^k}{k! e^\lambda}$.

For modelling price development, the assumption of constant jump size is rather unrealistic. In order to soften this requirement, a compound Poisson process can be used instead.

Definition 4.10 (Compound Poisson Process). *Assume $(Z_k)_{k \geq 1}$ an i.i.d sequence of square-integrable random variables, N_t as in Definition 4.9. Then, the process*

$$Y_t = \sum_{k=1}^{N_t} Z_k \quad (4.10)$$

is a compound Poisson process. See [32].

A classic version of a compound Poisson process would e.g. assume that the jump size is normally distributed.

Proposition 4.11 (Properties). *A few properties of the compound Poisson process which will be required later:*

- Similarly to Poisson process, the increments are independent
- $\mathbb{E}(Y_t) = \lambda t \mathbb{E}(Z_1)$
- $\text{Var}(Y_t) = \lambda t \mathbb{E}(|Z_1|^2)$

Proof. Rewrite Y as a sum over Z conditioned on $\{N_t = n\}$, the remainder is a standard calculation. $\square$

Definition 4.12 (Jump times). *Assume a Poisson process N_t . The first jump time T_1 fulfils the condition:*

$$\{T_1 > t\} \Leftrightarrow \{N_t = 0\} \quad (4.11)$$

i.e.

$$\mathbb{P}(T_1 > t) = \mathbb{P}(N_t = 0) = \exp(-\lambda t) \quad (4.12)$$

By applying this relationship for later intervals, it can be shown that

$$\mathbb{P}(N_t = n) = p_n(t) = \exp(-\lambda t) \frac{(\lambda t)^n}{n!} \quad (4.13)$$

and p_{n-1} is the density of T_n .

This definition will later become useful for sampling from a jump process.

In the next step, we want to combine the notion of a jump process with a Brownian Motion process as described in the section above.

Definition 4.13 (Jump diffusion process). *Assume N_t is a Poisson process with intensity λ and (Y_i) is a sequence of i.i.d. Gaussian random variables. Then, the process*

$$X_t = \mu t + \sigma W_t + \sum_{i=1}^{N_t} Y_i \quad (4.14)$$

is called jump-diffusion process.

A Lévy jump diffusion process can similarly be defined as:

$$X_t = \mu t + \sigma W_t + \sum_{i=1}^{N_t} Y_i - t\lambda i \quad (4.15)$$

, i.e. instead of a compound Poisson process, we use a compensated Poisson process (due to the martingale property).

To use the jump diffusion process to model prices, we again apply the transformation:

$$S_t = S_0 e^{X_t} \quad (4.16)$$

Such processes are known from the literature and used for example in the Merton model.

Mean reversion processes

A stochastic process which tends to drift towards a long-term mean is called mean-reverting. Mean-reverting processes are designed such that the further away the process gets from its mean, the higher the tendency to revert back.

Definition 4.14 (Ornstein-Uhlenbeck process). *The Ornstein-Uhlenbeck process [51] X_t is a stochastic process satisfying the SDE:*

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t \quad (4.17)$$

with $\theta, \sigma > 0$

The above process is widely used in Physics, Finance and many other areas. In Finance, Ornstein-Uhlenbeck process is used to e.g. model commodity prices. [54] μ represents a fundamental long-term equilibrium, σ is the degree of volatility and θ represents the rate at which the process reverts to its mean.

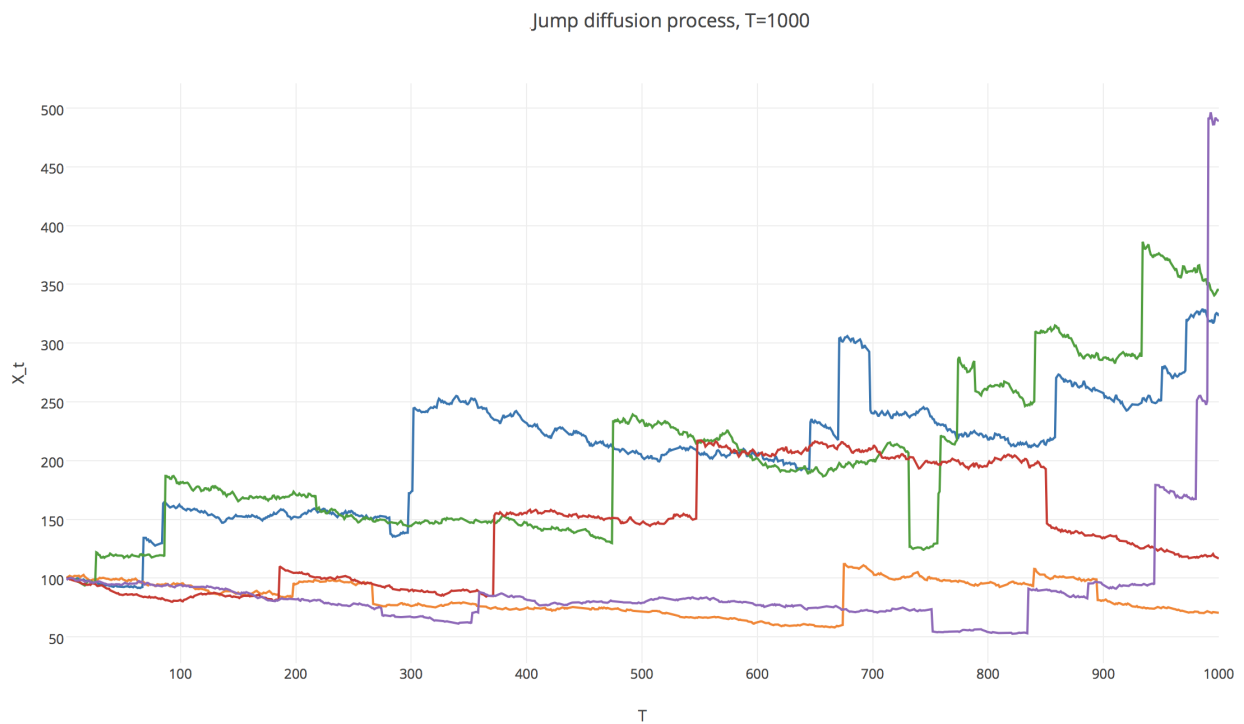


Figure 4.1: 5 samples of the jump diffusion stochastic process

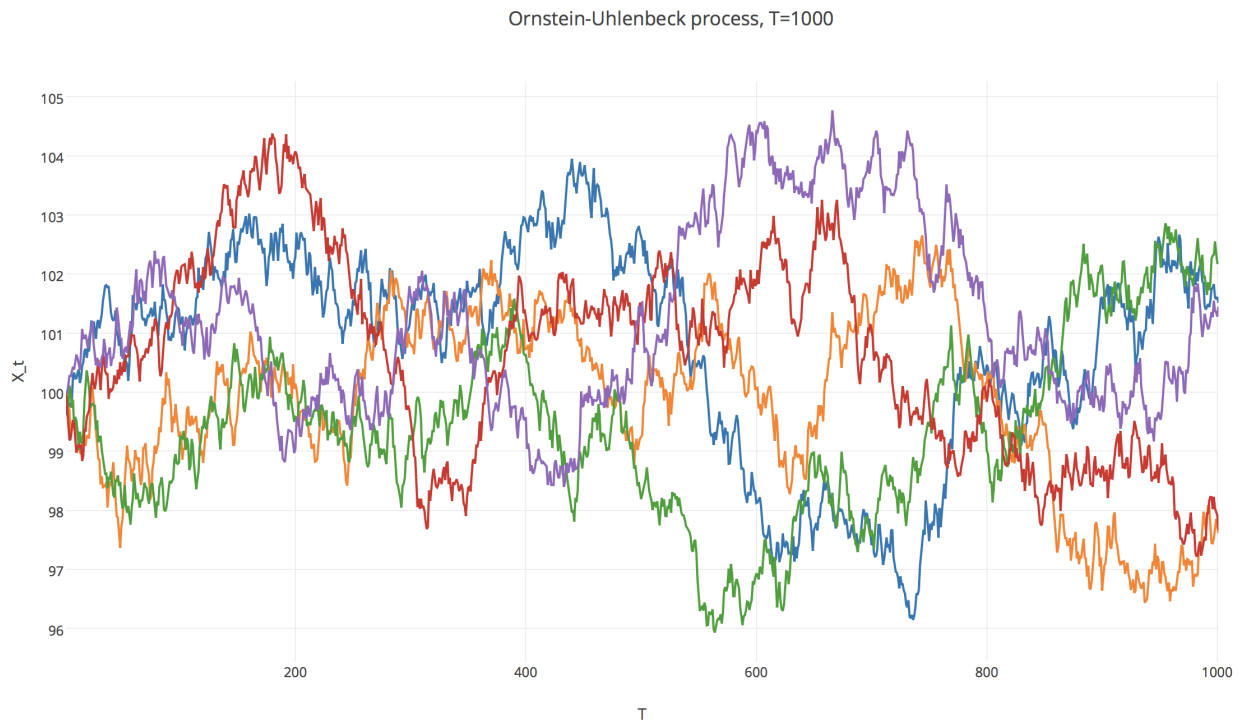


Figure 4.2: 5 samples of the Ornstein-Uhlenbeck stochastic process

Regime switching processes

One more relevant class of processes used to build stochastic programs is the class of regime switching processes. As an intuitive definition, a stochastic process is called regime-switching, if at different periods of time, its evolution is defined by a different model or a different set of parameters (regime). The process is allowed to switch randomly between different regimes. The switching behaviour is often described by a transition matrix, similar to the Markov process setting. This class of processes was first studied by [21]. One variation of regime-switching models implies a time series setting, where e.g. there is a parameter shift occurring at a random point of time. The process before the parameter shift is then described via a different AR/MA/ARMA model than after the shift.

Example 1. *A basic example of a regime switching model involves two possible regimes ($|R| = 2$) which are unobservable. Conditioned on the regime, the variable z_t is described as :*

$$z_t = \begin{cases} \alpha_0 + \beta z_{t-1} + \epsilon_t, & \text{if } s_t = 0 \\ \alpha_0 + \alpha_1 + \beta z_{t-1} + \epsilon_t, & \text{if } s_t = 1 \end{cases} \quad (4.18)$$

The variable s_t can either be modelled as a Bernoulli random variable independent of z_t or it can depend on z_t surpassing a certain threshold. For further reference, see [17].

For our modelling purposes, it is more interesting to study a shift between two continuous stochastic processes, e.g. a shift from GBM to another GBM with different parameters or a shift from GBM to a mean reversion model. For an overview of such models, see [10]. One of the standard variations, which will be used in this study is the following

Example 2 (Regime switching model). *Let $(Y)_t$ be a continuous-time Markov chain on finite space $\mathcal{S} = (1, \dots, K)$ for all $t \in [0, T]$. A regime switching, mean-reverting model is the solution of the SDE:*

$$dX_t = (\alpha(Y_t) - \beta(Y_t)X_t)dt + \sigma(Y_t)dW_t \quad (4.19)$$

where the functions α, β, σ assume values depending on the state of the Markov chain $(Y)_t$. The Markov chain $(Y)_t$ is considered exogenous and independent of the Brownian motion W_t .

Applying such models to real-world data requires an elaborate calibration, as discussed e.g. in [18] using an Expectation Maximization (EM) algorithm. For our purposes, we will just consider a set of already defined (realistic) parameters to sample from the process.

Heavy-tail processes

Definition 4.15 (Heavy-tail distribution). *Let X be a random variable with distribution function F . Its distribution is called right heavy-tailed, if*

$$\lim_{x \rightarrow \infty} e^{\lambda x} \mathbb{P}(X > x) = \infty, \forall \lambda > 0 \quad (4.20)$$

Same definition can be used to define left-heavy-tailed analogously.

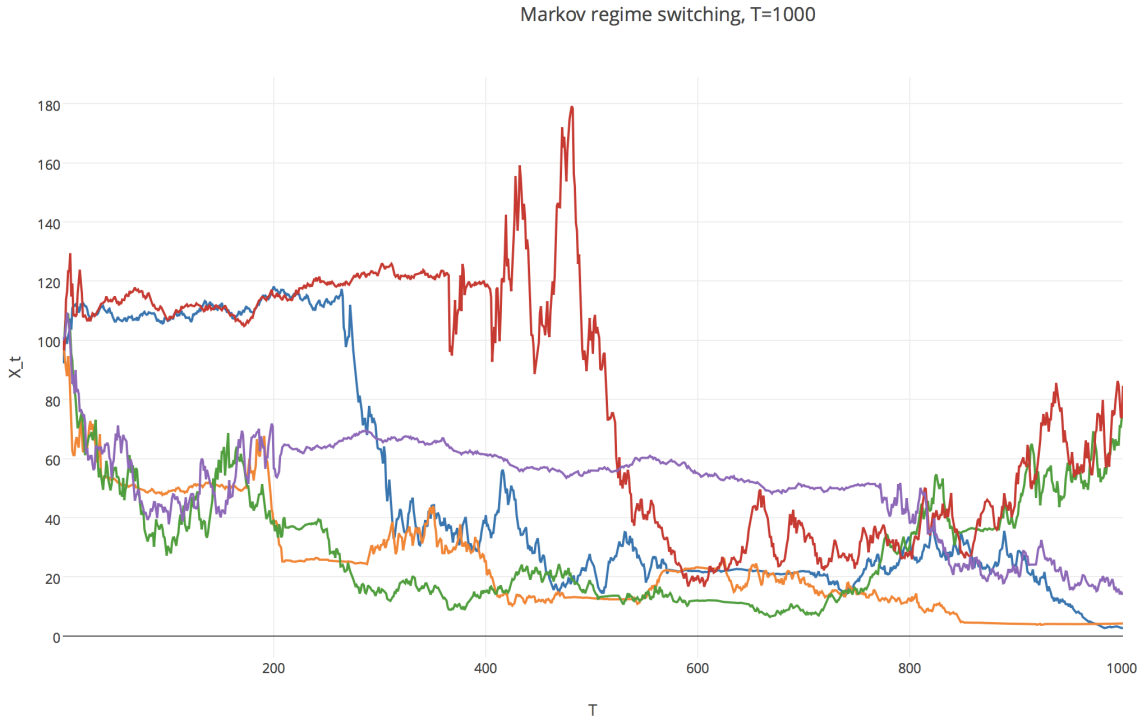


Figure 4.3: 5 samples of the regime switching stochastic process

The last class of processes we want to take a look at are those generated by a heavy-tail distribution, such as normal-inverse-Gaussian (NIG) for a certain parametrisation. NIG processes are widely used e.g. in Finance to model log-returns with properties such as:

- Heavy-tailed
- Skeweness
- Jumps
- Excess kurtosis

The NIG distribution was shown to fit the return processes of various financial instruments extremely well. [2]

Definition 4.16 (Inverse Gaussian distribution, IG). *Let W_t be defined as above, with $\mu > 0$ and variance σ^2 , $W_0 = w_0$. Consider the time τ until W_t reaches a level $z > w_0$ for the first time as a random variable. Then, $\tau \sim IG$. The density is given by:*

$$f(t, z, \mu, \sigma, w_0) = \frac{z - w_0}{\sigma \sqrt{2\pi t^3}} \exp\left(\frac{-(z - w_0 - \mu t)^2}{2\sigma^2 t}\right). \quad (4.21)$$

Definition 4.17 (NIG distribution). *The normal inverse Gaussian distribution with parameters $(\alpha, \beta, \mu, \delta)$, $NIG(\alpha, \beta, \mu, \delta)$ describes the variable X which satisfies:*

$$X|Z \sim N(\mu + \beta z, z), Z \sim IG(\delta, \sqrt{\alpha^2 - \beta^2}) \quad (4.22)$$

The parameters of NIG can be interpreted as follows:

- α controls the tails, for large values of α , the tails are light, for small values, they are heavy
- β controls the skewness of the distribution with $\beta = 0$ implying symmetric distribution around μ
- μ can be thought of as a location parameter (similar to the normal distribution)
- δ controls the scaling, i.e. the spread of values, similar to σ in the normal distribution case
- With $\beta = 0, \frac{\delta}{\alpha} = \sigma^2, \lim_{\alpha \rightarrow \infty} NIG(\alpha, \beta, \mu, \delta) = N(\mu, \sigma^2)$.

With these preliminaries, we can define the stochastic NIG process.

Definition 4.18 (NIG process). Assume $X_1 \sim NIG(\alpha, \beta, \mu, \delta)$. Then the process defined as

$$X_t = \mu + \beta Y_t + W(Y_t) \quad (4.23)$$

where Y_t is an IG-process, i.e. $Y_t \sim IG(\delta t, \sqrt{\alpha^2 - \beta^2})$ is called the NIG-process. It can also be seen as an inverse Gaussian Brownian motion with drift and changed time.

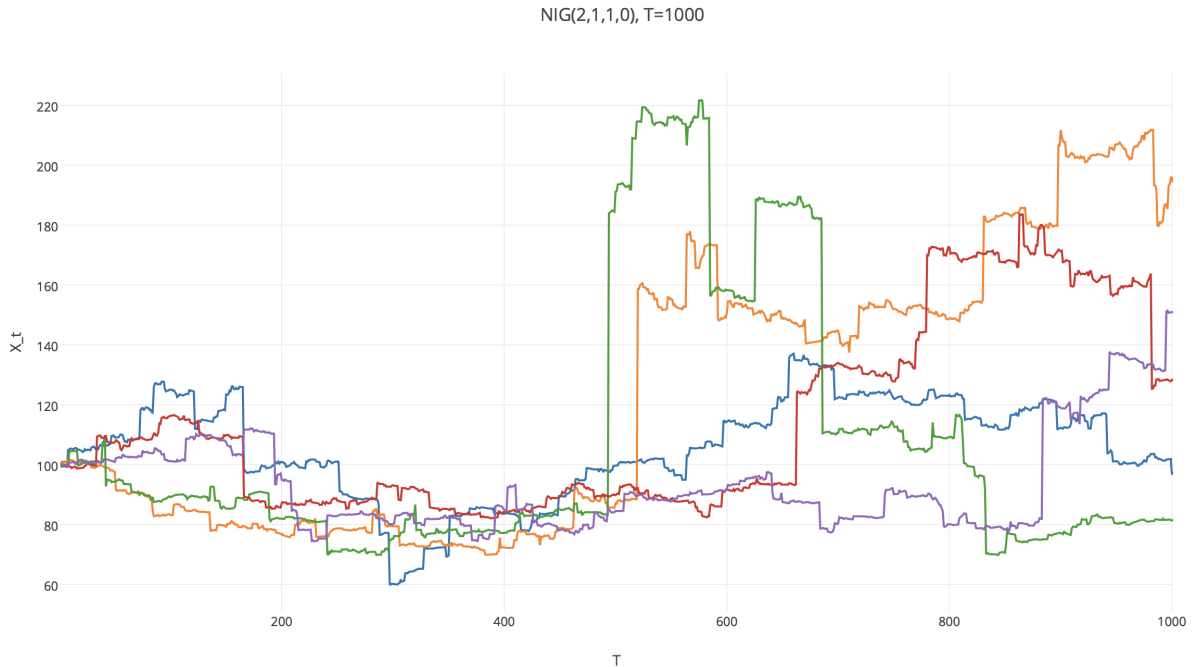


Figure 4.4: 5 samples of the NIG stochastic process

4.3 Selecting models

With stochastic problem classes identified in *section 4.1* and possible stochastic processes described in *section 4.2*, the next aim was to find well-known and relevant problems which vary with respect to the problem properties and the stochastic processes employed. The final choice fell on three models, each of them a classical one from the respective field. An overview of the selected models and how they relate to the general properties of stochastic programs can be found in the following table: Each of the mentioned problems is described

Model/Properties	APP	Flowergirl	ALM
randomness obj.	no	yes	no
randomness constr.	yes	yes/no	yes
# random variables	1-10	1-3	1-10
# stages	2-15	2-50	2-15
CVaR formulation	no	yes	no
stochastic processes	GBM,RS	GBM,JD,MR,RS,HT	GBM,MR

Table 4.1: Case study stochastic programs

in greater detail in its corresponding chapter.

4.4 Testing framework

The next step after identifying and selecting models for comparison is to establish the testing procedure. Our general aim of the testing is to compare the performance of ADDP-based algorithm for solving SPs to a scenario-tree based one. To allow a fair comparison, following conditions need to be met by the testing procedure:

1. **Scenario generation needs to be separated from scenario-tree/scenario-lattice construction.**

In order to generate scenarios, the first step is to sample from one of the stochastic processes described above. For some of the simpler processes (e.g. GBM), sampling can take place directly by e.g. drawing from the Normal distribution. For other processes (e.g. NIG), techniques such as Antithetic sampling [6] and Acceptance-Rejection [11] are employed.

Moreover, in order to simulate correlated random variables conveniently, a random correlation Matrix needs to be drawn. This allows to control the degree of correlation via one parameter, as opposed to constructing a (potentially big) correlation matrix manually. For drawing random matrices, we follow the approach presented in [29], [34]. As can be seen from *Figure 4.5*, depending on one parameter, the overall degree of correlations can be adjusted.

2. **In order to sample from stochastic processes, common random numbers need to be used to construct both, the scenario-tree and the scenario-lattice.**

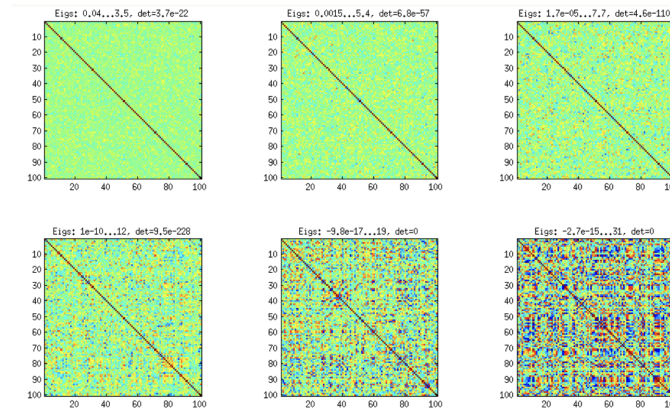


Figure 4.5: degrees of correlation visualised

This requirement is achieved by generating scenarios in Matlab and then passing them on as matrices to the tree/lattice construction libraries. In cases where the tree-construction library was not able to handle the total amount of paths from a process, the biggest possible random subset was passed.

3. State-of-the-art algorithms need to be employed for both approaches.

The current state-of-the-art algorithms for constructing and reducing scenario trees are based on the work presented in [43],[13],[23]. Moreover, these methods are a part of a GAMS based library called SCENRED. For lattice-construction and ADDP, we employ a library called Quasar, implemented in Java 7 with the linear programs being solved by Sulum 4.3. Both will be described in more detail in the *section 4.6*.

4. The comparison needs to be fair in terms of runtime, i.e. the maximum time allowed to solve an instance of a SP is set to be the same for both approaches.

We measure the time required to solve a given instance of a SP and scale the precision of the algorithms (i.e. number of nodes in a tree or number of iterations in ADDP) such that the maximum available time is used up. The time limit for the tests was 1 day runtime for solving one instance of a SP.

5. Avoid random effects by solving SPs multiple times and averaging the payoffs and the decisions

Since both, the payoff and the decisions made are random variables (depend on the paths sampled for a particular run), we compare the performance of both approaches over multiple runs by averaging the generated payoff and the decisions. Averaging will be described in more detail in *section 4.5*.

6. Comparison criteria need to represent a real-world application as precisely as possible.

Instead of comparing theoretical measures (such as VSS) and first-stage-decisions only, we create a framework for comparing the solutions and the policies out-of-sample.

7. The sets of problem parameters need to be representative for the particular problem class.

For each of the identified SPs, we use the parameter sets which are as close to the respective problem as possible.

8. Relatively complete recourse

All of the problems used for testing need to fulfil the relatively complete recourse property as described in *Definition 2.6*.

4.5 Comparison criteria and statistical tests

In this section, we present our criteria for comparing solutions generated by solving the deterministic equivalent over a scenario-tree as opposed to applying ADDP to a scenario-lattice.

1. Standard measures

The most trivial approach to comparing solutions is to compare the expected payoffs generated by each method. While this already provides an insight on the relative performance, the results can be biased due to the in-sample calculation, i.e. the *same* sample of the random process is used for both, calculating projected payoff *and* optimizing the decisions. In particular, it is not clear whether the projected payoff would have actually occurred, given that the true process continues following the distribution until the last stage. The next approach is to calculate the mean-value-solution (averager solution) as described in *section 2.3* and compare the stochastic solutions generated by each method to the former. However, this, in turn, does not provide any insight on the actual performance of each method.

2. Rolling horizon approach

The idea behind the rolling horizon approach is depicted in *Figure 4.6*. First, in step **(a)**, random scenarios are drawn from the distribution of the underlying stochastic process. Then, in **(b)**, a scenario tree is constructed based on these samples. The planning/optimization then takes place based on the constructed tree for all future stages. This is done iteratively, until the last stage of the stochastic program is reached. At stage $T - 1$, the last stochastic two-stage program is solved **(c)**. Lastly, at stage T , a deterministic evaluation of the program takes place. This solution can be considered as being in-between here-and-now and wait-and-see types and the payoff of this solution provides a viable measure for comparison.

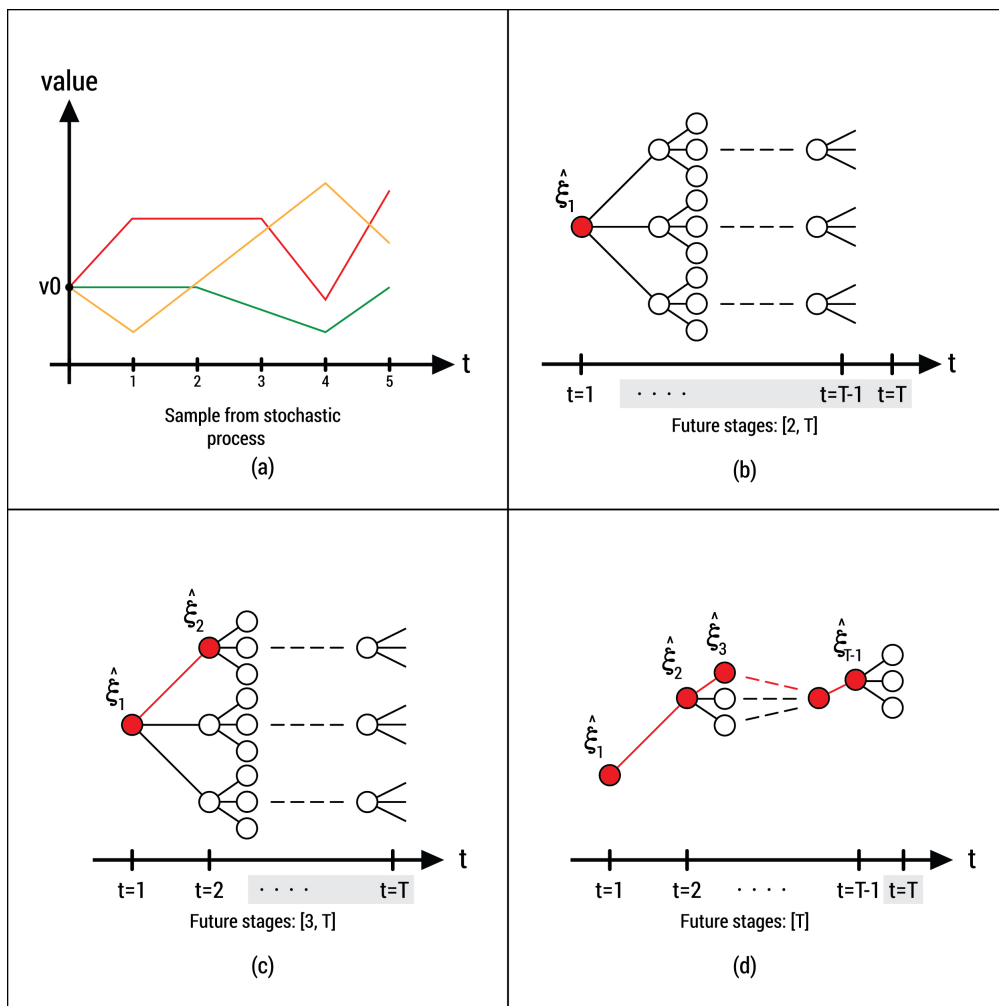


Figure 4.6: A schematic representation of the rolling horizon evaluation procedure

3. Out-of-sample planning approach

General idea: *Initial first-stage optimal decisions are tested on a different set of scenarios than the ones used to find the optimal decisions.* This allows to avoid over-fitting and provides a more "real" assessment of the first-stage decision quality. The out-of-sample planning approach is explained schematically in Figure 4.7. This approach combines the idea of a rolling horizon with an out-of-sample procedure. For each of the stages from 1 to T , a new sample of the underlying stochastic process is drawn independently of the previous ones. The procedure thus starts at stage 1, generates a sample and builds a scenario-tree (or scenario-lattice analogously). Subsequently, the planning takes place for all stages until T . In the next step, the first-stage decision found in step 1 is being implemented (e.g. if there was a decision on buying assets or hiring labor force, we assume that this actually took place). Then, we simulate one state transition based on the stochastic process and draw a new sample from the process, given the realized transition. The sample is being drawn for all stages $[2, T]$. After that, the optimization takes place again. This is repeated iteratively until stage $T - 1$. The payoffs and the first-stage-decisions are stored along the way. The result of this procedure is a vector consisting of $T - 1$ payoffs (or costs) generated by the corresponding stochastic programs and one final payoff (cost) at time T , which is calculated deterministically (e.g. selling all assets at the end of the planning horizon). Apart from that, for each decision x^i , we store a vector containing the values for each of the $T - 1$ planning stages.

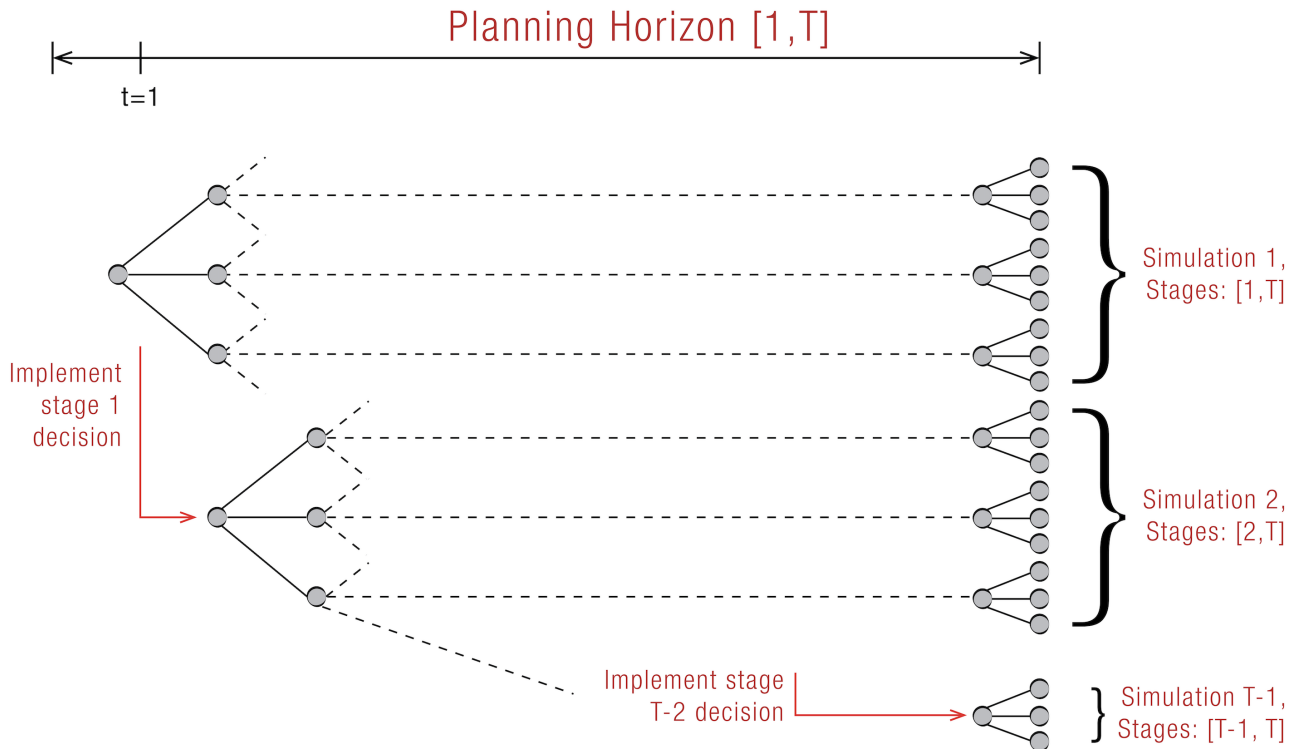


Figure 4.7: A schematic representation of the out-of-sample rolling planning procedure

4. Comparison of out-of-sample payoffs

Now, based on the idea described above, we run the out-of-sample procedure for both, the tree-based method and ADDP. For each instance, the output will be:

$$\pi^{tree} = (\Pi_1^{tree}, \dots, \Pi_T^{tree}), \quad (4.24)$$

$$\pi^{addp} = (\Pi_1^{addp}, \dots, \Pi_T^{addp}) \quad (4.25)$$

where Π denotes either the payoff or the costs associated with the stochastic program. In order to compare these two (random) vectors, we repeat the planning N times, to get a $N \times T$ matrix of payoffs:

$$\Pi^{tree} = \begin{pmatrix} (\pi_1^{tree})_1 & (\pi_2^{tree})_1 & \dots & (\pi_T^{tree})_1 \\ \vdots & \vdots & \vdots & \vdots \\ (\pi_1^{tree})_N & (\pi_2^{tree})_N & \dots & (\pi_T^{tree})_N \end{pmatrix} \quad (4.26)$$

and

$$\Pi^{addp} = \begin{pmatrix} (\pi_1^{addp})_1 & (\pi_2^{addp})_1 & \dots & (\pi_T^{addp})_1 \\ \vdots & \vdots & \vdots & \vdots \\ (\pi_1^{addp})_N & (\pi_2^{addp})_N & \dots & (\pi_T^{addp})_N \end{pmatrix} \quad (4.27)$$

Lastly, we calculate:

$$\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^T (\Pi^{tree})_{i,j} \quad (4.28)$$

and

$$\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^T (\Pi^{addp})_{i,j} \quad (4.29)$$

along with the standard error:

$$\frac{1}{\sqrt{N}} \sigma \left(\sum_{j=1}^T (\Pi^{tree})_{i,j} \right) \quad (4.30)$$

and

$$\frac{1}{\sqrt{N}} \sigma \left(\sum_{j=1}^T (\Pi^{addp})_{i,j} \right) \quad (4.31)$$

5. Comparison of objective value deviations

Apart from comparing the out-of-sample payoffs over N instances, we are also interested in the deviation of the out-of-sample payoff from the in-sample payoff calculated when planning for the whole horizon $[1, T]$. Ideally, the deviation should be as small as possible, indicating that the discretization of the underlying process is precise.

6. Comparison of policy stability

For all stages, we can also compare how much the planned later-stage policy deviates from the actual later-stage policy evaluated when the planning progresses until the corresponding stage.

7. Statistical tests

Lastly, since out-of-sample payoffs are random samples, drawn from the "population" of payoffs N times, we can apply statistical testing methods to check whether the difference in the distribution of the payoffs is significantly different between tree-based payoffs and ADDP-bases ones. In particular, we can test for equality of means of these two populations.

The first possible test is a paired-t-test, checking whether the true mean of the difference between both summed payoffs (μ_d) is equal to zero, i.e.

$$H_0 : \mu_d = 0 \tag{4.32}$$

$$H_1 : \mu_d \neq 0 \tag{4.33}$$

We therefore pass two vectors of the length N , containing the payoffs for the tree-based method and for ADDP and calculate the p-value of the test, rejecting H_0 for $p \leq 5\%$.

An alternative to this test is a non-parametric one, e.g. the Wilcoxon Signed-Rank Test. This is especially relevant for small N . All of these values will be reported in the coming chapters when we evaluate the results.

4.6 Technical setup

For the theoretical testing framework described above, an appropriate technical setup had to be created. The challenges of the technical setup included:

- Separating scenario generation from scenario-tree / scenario-lattice construction
- Using common random numbers to generate scenarios for both, the tree and the lattice
- Building up on existing state-of-the-art library for scenario tree construction and reduction (GAMS/SCENRED)
- Building up on the Quasar library for lattice construction and ADDP solving
- Linking all libraries together in MATLAB
- Modelling and result evaluation in MATLAB

In order to build up the testing setup according to this criteria, a few interim steps had to be made.

Porting Quasar Java library into Matlab

Since Quasar is only available as a Java library (combined with an existing Python interface), a first step is to make this library accessible from Matlab. Porting the library has several advantages: The time effort for modelling in Matlab is highly reduced due to a higher abstraction level and more natural, mathematical model formulation. The corresponding formulation in Java is rather lengthy, particularly since all instances of all classes have to be initiated and parametrized explicitly. Moreover, having the Java library accessible from Matlab allows to easily combine basic functionalities (e.g. matrix manipulation, sampling from distribution etc.) with more advanced functionalities from the Java library. In addition to that, since all Java classes are being wrapped into the corresponding Matlab classes, there is an advantage of combining Matlab-specific methods with Java-specific classes, e.g. it becomes possible to generate a data matrix in Matlab and pass it on to Java, thus saving the effort of initializing and building a Java matrix object. In the following, an example of a small aggregate production planning model is used to compare the model building in Matlab vs. the model building in Java. As can be seen by comparing both code examples, the Matlab-wrapped version of Quasar allows for a significantly more "natural" formulation of the optimization problems, including the usage of operators as $+$, $-$, $\geq$, $\leq$. Moreover, objects don't need to be explicitly created. Instead, they are being created in the background "on the fly".

Hardware setup

All numerical studies were run on a Core i7 2.4GHz computer with 16GB of RAM. The maximum time allowed for solving one instance of a given stochastic program was limited to 1 hour.

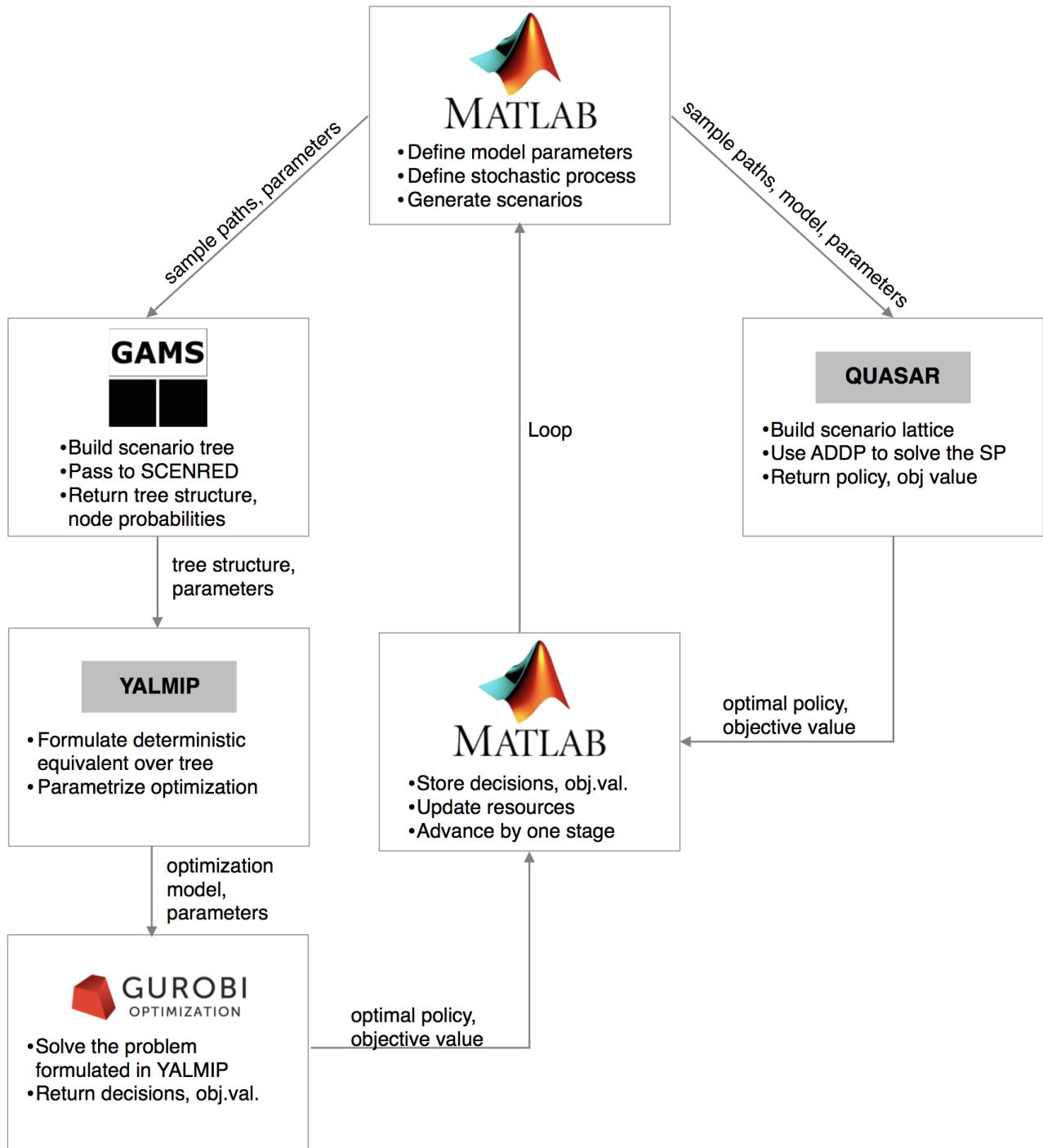


Figure 4.8: Overview: technical side of the rolling stochastic procedure

Trees/Lattices size and sampling

For all of the problems mentioned below, the amount of samples drawn from the stochastic process was between $5e5$ and $2e6$. The amount of nodes in the tree was chosen as high as possible, under the constraint that the maximum runtime is not exceeded. In most cases this meant between $5e4$ and $2e5$ leaf nodes. For lattices, the amount of nodes per stage varied between 50 and 200.

Chapter 5

Stochastic aggregate production planning (APP)

Aggregate planning aims to optimize future operational plans for a given company under demand uncertainty.

In general, a company is facing a tradeoff between satisfying uncertain demand and producing at the right capacity levels and is therefore trying to balance between these two restrictions. Violating the demand restriction would result in reduced revenues, while employing too much capital to meet every possible demand would result in run-away costs. The aim of such planning is therefore to minimize the expected production costs or to maximize the expected revenue.

These problems arise mostly for intermediate-term planning, as opposed to short or long-term planning and are thus encompassing a timespan of at most 18 months.

The term "aggregate" refers to the fact that the whole span of resources (e.g. materials, workers) and the whole span of production are aggregated into meaningful units, such as total amount of hours worked, total amount of material employed, total volume of output and others.

Various strategies for managing uncertain demand arise in this context, such as: Building up inventory to serve over-demand, employing varying capital depending on the current forecast for the next period, using subcontractors and many more. All of these decisions can be modelled as a part of an APP problem to arrive at an optimal strategy.

In the following, an aggregate production planning model as introduced by [5] will be discussed. In this model, the decision maker is facing a problem of uncertain demand for i different items. In order to produce these items, certain labor hours are required. The available labor is subdivided into regular and overtime, with the last one being more expensive. The decision on regular labor is met once in the first stage and the amount of hired workers stays constant over all stages. According to the classification from *section 4.1*, the given formulation of APP contains a random variable in the constraints only (in this case, random demand), which is modelled by a GBM process. Moreover, since the production planning takes place for n products, the number of random variables per stage can vary.

A few observations regarding this model:

- Stochastic problem with scalable dimension (number of products)
- Two types of decisions
 - Inventory and production : in anticipation of the next stage
 - Regular labor: first-stage decision only
 - Overtime labor: Can be decided and ordered to ensure problem feasibility, i.e. even for unexpectedly high demand and underestimated regular labor force, the production can still ramp up to meet the demand

5.1 Problem definition

In the following, an instance of an APP problem is defined using the following constants and variables:

- c_{it} : unit production costs excluding labor
- h_{it} : inventory carrying cost
- m_i : hours required to produce one unit
- r_t, o_t : regular and overtime labor costs
- D_{it} : random demand for each of the i products at time t
- R_1 : regular labor hiring decision **made at stage 1 for all stages**
- O_t : overtime labor hiring decision
- X_{it} : production decision for each of the i products
- I_{it} : inventory decision for each of the i products
- c^R : total cost of regular labor

Using these definitions, the aggregate production problem can be stated as follows:

$$\begin{aligned} \min \quad & \sum_{i=1}^I c_{i,1} X_{i,1} + h_{i,1} I_{i,1} + c^R + o_1 O_1 + \\ & + \mathbb{E} \left[\sum_{t=2}^T \sum_{i=1}^I c_{it} X_{it} + h_{it} I_{it} + c^R + o_t O_t \right] \end{aligned}$$

subject to

$$I_t - I_{t+1} + X_{it} = D_{it} \quad \forall t \in [1, \dots, T-1] \quad (5.1)$$

$$I_{i,T-1} + X_{i,T} = D_{i,T} \quad (5.2)$$

$$\sum_{i=1}^I m_i X_{it} \leq R_1 + O_t \quad \forall t \in [1, \dots, T] \quad (5.3)$$

$$R_1 \leq b \quad (5.4)$$

$$X_t, I_t, R_t, O_t \geq 0 \quad \forall t \in [1, \dots, T] \quad (5.5)$$

The constraints (1),(2) enforce that the demand for each of the i products is met in each period, including the last-stage demand $D_{i,T}$ met at the last stage.

Constraint (3) ensures that the sufficient amount of workforce is employed to produce each of the i products. The total work force consists of regular work-force, decided upon in stage 1 and overtime workforce, which can vary across the stages. Constraint (4) ensures that the maximum amount of employable regular workforce is bounded and constraint (5) ensures that all decision variables are non-negative.

5.2 Modelling random demand for products

For each of the i products, the random demand process $D_{i,t}$ is modelled as follows: $\forall(i, t) : D_{it} = D_{i0} \exp((\mu_i - \frac{\sigma_i^2}{2})t + \sigma_i W_t^i)$, W_t^i Wiener processes with drifts μ_i , volatilities σ_i . Further, $(\Sigma)_{i,j} = Cov(D^i, D^j)$. We draw Σ randomly as proposed by [34].

→ GBMs are approximated by drawing between 50.000 and 1.000.000 realizations, depending on the problem size.

The demand GBM-processes are further assumed to have no drift, i.e. $\mu = -\frac{\sigma^2}{2}$, with $\sigma \in [0.01, 0.2]$. The volatility is adjusted to the number of stages, such that in the extreme cases, the demand does not exceed a certain multiple on average, in our case, the demand should not triple of the course of the planning horizon. Moreover, a random correlation matrix is drawn to reflect an interdependency of demand between the products. A negative coefficient represents substitute products, i.e. a high demand for product 1 means a lower demand for product 2. A positive coefficient represents complementary products, i.e. both demands go either up or down approximately synchronously. The level of dependency is described via one parameter of dispersion, where the dispersion is a measure of how much the correlation matrix deviates from a unit matrix. For further references on random correlation matrix generation, see *section 4.4*.

Moreover, other parameters of the problem are given by:

$$\begin{aligned} c_{i,t} &\in [1, 5] \\ h_{i,t} &\in [0.1, 0.5] \\ m_i &\in [3, 4] \\ r_t &\in [5, 10] \\ o_t &\in [15, 25] \end{aligned}$$

The study is conducted by varying the number of stages in the planning horizon, the number of random variables as represented by the products and the degree of dependency between the demands, as described by the dispersion parameter mentioned above.

5.3 Numerical results

T	Dependency	Costs Tree	Costs Lattice	Diff %	t-paired test
5	very high	12432	9957	19.91%	<1%
5	high	80095	70307	12.22%	<1%
5	medium	79129	70254	11.22%	<1%
5	low	79263	70230	11.40%	<1%

Table 5.1: Costs for varying dependency between products

numProd	Costs Tree	Costs Lattice	Diff %	t-paired test
1	12474	11253	10.85%	<1%
3	3084	2739	12.57%	<1%
5	7974	7040	13.26%	<1%
7	16275	14315	13.69%	<1%
9	17558	11745	49.49%	<1%

Table 5.2: Costs for varying number of products, T=5, medium dependency

T	Costs Tree	Costs Lattice	Diff %	t-paired test
3	4466	4146	7.15%	2.8%
5	8009	7066	11.78%	<1%
10	15843	10382	34.47%	<1%

Table 5.3: Costs for varying planning horizon length

Again, we observe that with increasing total dimension of the stochastic program, the gap between tree-based and ADDP-based solutions increases. Planning using ADDP clearly helps the decision-maker to save costs.

By comparing the actual decisions, we can find a reason for this discrepancy. For example looking at production decisions over 5 periods and 30 samples and plotting the differences between units produced in a tree-based policy and units produced in an ADDP-based one, we get a heatmap presented in *Figure 5.2*. The blue color represents cases where the tree-based policy is producing more, while the red color represents the opposite. Thus, we observe that while both policies are able to meet the same demand, the tree-based one is producing more, partly due to an inefficient usage of storage capacities. The overproduction is usually carried out by more expensive overtime labor force, causing an increase of total costs.

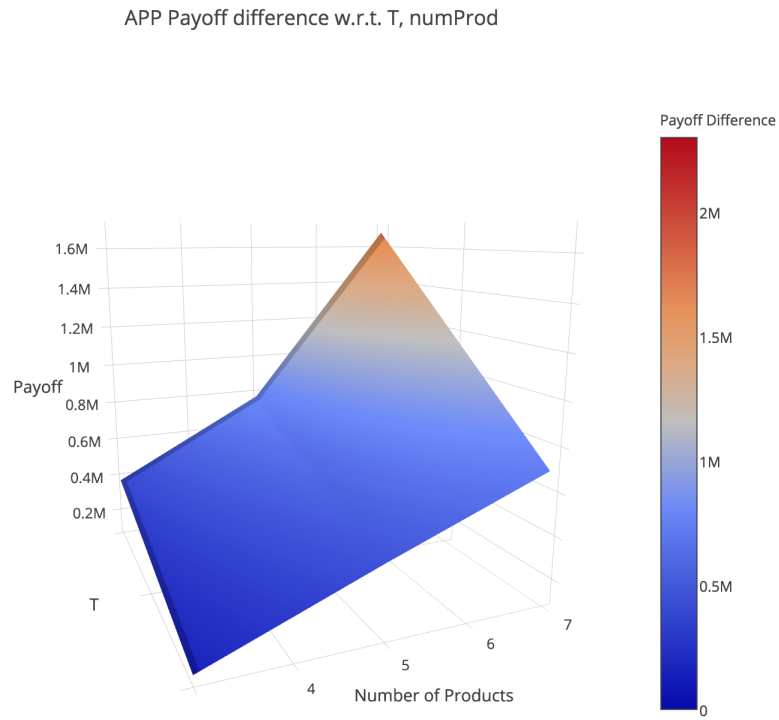


Figure 5.1: Payoff difference depending on T, numProd

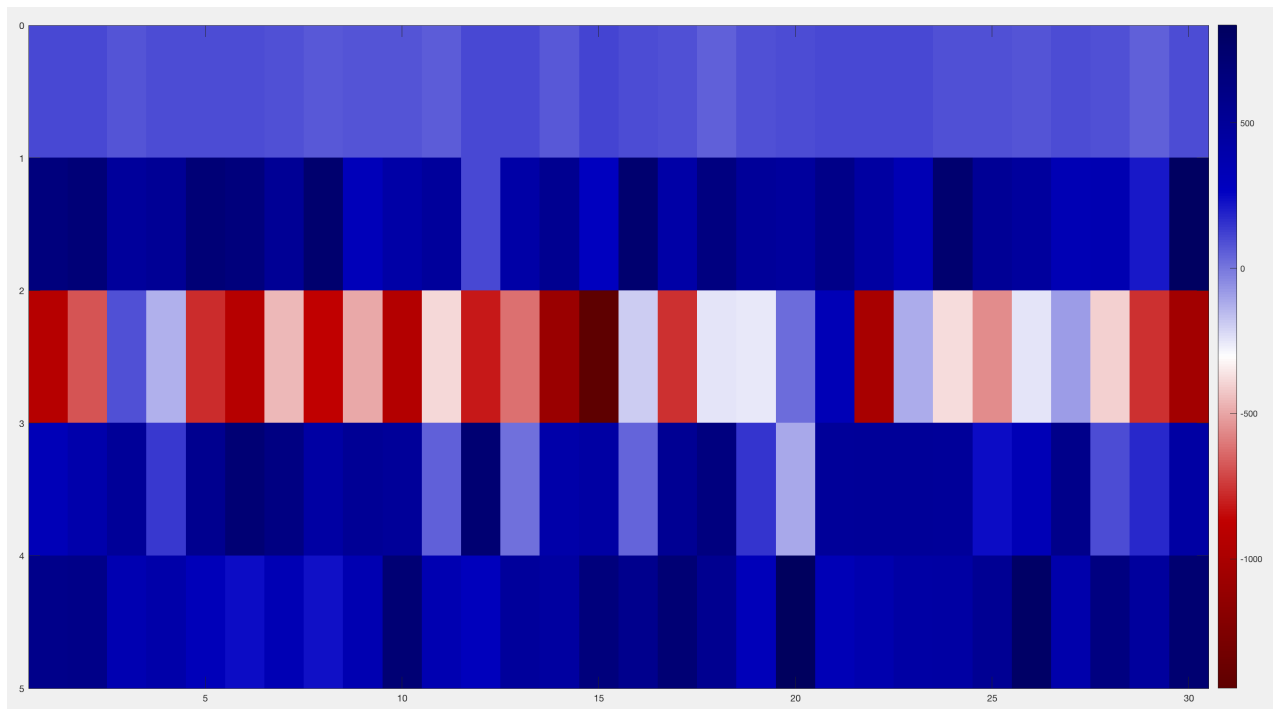


Figure 5.2: Production decisions difference heatmap (T=5)

Chapter 6

The flower girl optimization problem

The flower girl problem is a classical multistage stochastic optimization problem. At each stage, the flower girl is facing a random demand for flowers. To fulfil this random demand, she has to order flowers one stage before selling them. Ordering too many flowers would result in losses due to storage limits (physical limit as well as a time limit, since flowers fade over time). Ordering too little would however result in capital opportunity costs. The idea is thus to order the amount balancing these two restrictions. Characteristics of the basic flower girl problem:

- Simple problem with only one random variable
- Two types of decisions:
 - Inventory and ordering: in anticipation of the next stage
 - Selling: realized in the same stage

According to criteria from *section 4.1*, the basic flower girl problem can be easily modified, for example

- Apart from random demand, the ordering and the selling prices can be random (higher dimensionality)
- The planning horizon can easily be extended to arbitrarily many stages
- The underlying stochastic process can be changed to each of the processes from *section 4.2*.

6.1 Standard problem definition

In the following, an instance of a flower girl problem is defined using the following constants and variables:

- p^R, p^S : Ordering / selling prices
- c : Stored items fading factor
- b : Maximal storage capacity available
- D_t : Random demand in stage t
- I_t : Inventory in stage t
- R_t : Ordering decision
- S_t : Selling decision
- In_t : Amount of flowers put into storage in state t
- Out_t : Amount of flowers taken out of storage in state t
- $Disc_t$: Amount of flowers discarded in state t

Using these definitions, the flower girl problem can be stated as follows:

$$\max p^S S_1 - p^R R_1 + \mathbb{E} \left[p^S \sum_{t=2}^{T-1} S_t - p^R \sum_{t=2}^{T-1} R_t \right]$$

subject to

$$I_{t+1} = cI_t - Out_t + In_t - Disc_t \quad \forall t \in [1, \dots, T-1] \quad (6.1)$$

$$Out_t + R_{t-1} = S_t + In_t \quad \forall t \in [2, \dots, T] \quad (6.2)$$

$$S_t \leq \min(I_t, D_t) \quad \forall t \in [1, \dots, T] \quad (6.3)$$

$$I_t \leq b \quad \forall t \in [1, \dots, T] \quad (6.4)$$

$$S_t, I_t, R_t \geq 0 \quad \forall t \in [1, \dots, T] \quad (6.5)$$

The evolution of random demand D_t over time can be simulated via different stochastic processes. In the most basic case, D_t is assumed to follow a geometric Brownian Motion with a certain drift μ and a volatility σ . D_0 will be fixed to a certain value.

6.2 Problem variations

The standard flower girl problem as described above can be further modified to reflect varying sources of randomness and varying forms of the objective:

Random selling price

Assume the same form as described in the basic definition above and let D_t and P_t^S both follow a random process, e.g. two (possibly correlated) geometric Brownian Motions $\forall t$:

$$\begin{aligned} D_t &= D_0 \exp\left(\left(\mu_1 - \frac{\sigma_1^2}{2}\right)t + \sigma_1 W_t^1\right) \\ P_t^S &= P_0^S \exp\left(\left(\mu_2 - \frac{\sigma_2^2}{2}\right)t + \sigma_2 W_t^2\right) \end{aligned}$$

With W^1, W^2 two Wiener processes, possibly correlated with correlation coefficient ρ .

Random selling and ordering prices

Assume the same form as above and let the ordering price P_t^G be random, i.e. $\forall t$:

$$\begin{aligned} D_t &= D_0 \exp\left(\left(\mu_1 - \frac{\sigma_1^2}{2}\right)t + \sigma_1 W_t^1\right) \\ P_t^S &= P_0^S \exp\left(\left(\mu_2 - \frac{\sigma_2^2}{2}\right)t + \sigma_2 W_t^2\right) \\ P_t^G &= P_0^G \exp\left(\left(\mu_3 - \frac{\sigma_3^2}{2}\right)t + \sigma_3 W_t^3\right) \end{aligned}$$

With W^1, W^2, W^3 being three Wiener processes, possibly correlated with correlation coefficients ρ_1, ρ_2 .

Risk-averse planning

Assume the same model as in (5.2.1). The objective function is modified to reflect risk-averse planning, i.e. a new variable $cumProfit_t$ and a new last-stage decision variable $shortFall_T$ are added, such that

$$shortFall_T \geq var - cumProfit$$

where $cumProfit$ represents the cumulative profit up to stage t and var represents the quantile. Moreover, for γ and α , the objective of the optimization problem is modified as follows:

$$\max (1 - \gamma)cumProfit_t + \gamma\left(var - \frac{1}{\alpha}shortFall_T\right)$$

Higher stage-dependency due to $cumProfit$ and $shortFall$ decision variables. Risk-averse planning in the objective.

Random selling price with jumps

In this variation, the stochastic process modelling the price evolution is allowed to have jumps. Thus, $P_t^S \sim X_t$, where X_t is a jump process as described in *section 4.2*. The parameters are set as follows:

$$\begin{aligned}\lambda_J &= 10 \\ \mu_J &= 0.2 \\ \sigma_J &= 0.3 \\ \lambda &= 3\end{aligned}$$

where $*_J$ describe the parameters of the jump process, i.e. jump frequency, mean jump size, mean jump volatility.

In all of the variations, the demand process is assumed to have no drift, i.e. $\mu = -\frac{\sigma^2}{2}$, with $\sigma \in [0.1, 0.5]$. Sigma is adjusted to the number of stages, such that in the extreme cases, the demand does not exceed its initial value by 3 times. In case of correlation between demand and price, ρ is set to 0.3. For random demand, ordering price, selling price, $\rho_1 = 0.3$, $\rho_2 = 0.7$

Moreover, other parameters of the problem are given by:

$$\begin{aligned}p^R &= 3 \\ c &= 0.2 \\ b &= 1000 \\ \text{maxOrder} &= 7000\end{aligned}$$

6.3 Numerical results

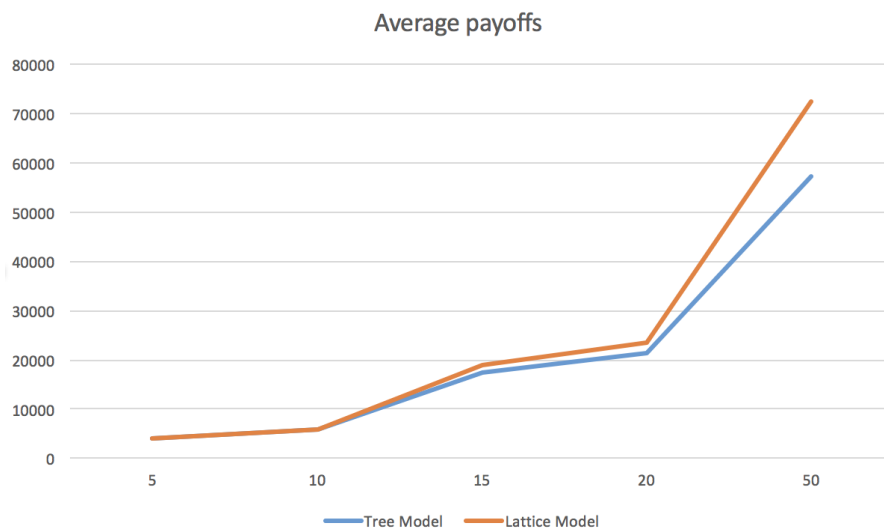


Figure 6.1: Comparison of payoffs, basic flower girl model

T	Profit	stderror	ProfitA	stderrorA	Diff %	t-paired test
5	3919.8	86.7	3925.6	85.9	0.15%	—
10	5743.5	117.3	5945.1	112.1	3.51%	—
15	17297.2	9741	18940.3	9165.3	9.50%	4.1%
20	21303.3	8331.2	23423.1	7431.3	9.95%	4.1%
50	57343.3	12122.2	72543.2	8331.2	26.51%	2.0%

Table 6.1: Comparison of payoffs, basic flower girl model

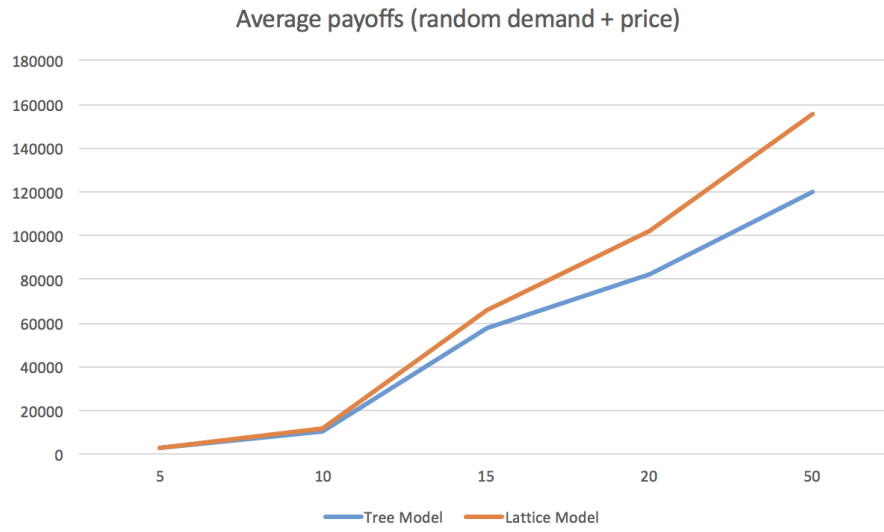


Figure 6.2: Comparison of payoffs, random demand and random price flower girl model

T	Profit	stderror	ProfitA	stderrorA	Diff %	t-paired test
5	2973.1	372.1	3180.3	323.3	6.97%	—
10	10508	1361.6	11813.5	1301.4	12.42%	—
15	57389.1	3822.8	66122.7	3012.2	15.22%	—
20	82332.3	4520.1	102433.4	4001.2	24.41%	5,1%
50	120321.3	15339.21	155312.8	9342.2	29.08%	1,0%

Table 6.2: Comparison of payoffs, random demand, random price flower girl model

T	Profit	stderror	ProfitA	stderrorA	Diff %	t-paired test
5	2936.8	459.2	3143.5	455.1	7.04%	—
10	9284.2	1880.1	10675.4	1950.1	14.98%	—
15	57893	3829.2	71050.7	4015.2	22.73%	4.5%
20	79871.1	5733.9	103212.3	4783.1	29.22%	2.1%

Table 6.3: Comparison of payoffs, CVaR variation, $\alpha = 0.01, \gamma = 0.5$

The numerical result show that for smaller instances of the flower girl problem, there is no significant difference between the tree-based solution and the ADDP-based one. However, for bigger instances and in particular for more complex problem formulations (random price, CVaR), the outperformance of ADDP-based policy becomes clear.

For varying stochastic processes, it can be observed that irregardless of the process, for small T , there is no significant difference of mean profits between tree-based and lattice-based methods. For $T > 10$, the flower girl problem with jump process and NIG process as defined in *section 4.2* is solved by ADDP much better than by the tree-based methods, with payoff difference being significant at about 30%.

Chapter 7

Stochastic asset liability management problem (ALM)

Asset liability management is a well-known problem in the Finance industry. The aim of ALM is to buy and sell financial securities such that the liabilities can be met at any time. This problem is mainly faced by asset managers of pension funds and insurance companies. On the one hand, such companies receive payments, either in form of pension payments or in form of insurance premium payments. On the other hand, they also face liabilities, either by e.g. pensions being claimed or by insurance claims which have to be covered. The accumulated funds have to be managed to maximize risk-adjusted return during the time when no claims are made. A vast variety of models has been developed in the literature to deal with such problems. The main difficulty of ALM is the randomness coming from different sources:

- Random liabilities
- Random interest rates
- Random security prices

All three sources of randomness are usually described by a different stochastic model. The next difficulty lies in the selection of an appropriate model for a given use-case. Apart from that, numerical tractability becomes a problem with realistic cases, especially when the decision-maker has many possible assets to invest in and a complex liabilities process. Despite the difficulties, dynamic LP models for asset liability management exhibit various advantages, as described by [48] :

- LP models can make specific recommendations at the individual asset level to actively leverage assets to enhance net worth (objective) rather than simply evaluating a static recommendation such as simulation approaches or providing only broad recommendations in terms of asset classes such as stochastic control methods.
- Meeting regulatory requirements and liquidity requirements (constraints) and using the output of auxiliary interest-rate and cash-flow models as input is straightforward with LP

- LP can incorporate "imperfections", including taxes, transaction costs, and regulatory or institution-specific requirements over time more flexibly than other approaches
- Recent advances in LP solver technology and in stochastic programming also help make the case for LP.

Despite these advantages, dynamic LP models are considered complicated and their benefits are questioned by the industry. This is mainly due to the fact that solving realistic instances in combination with dynamic recourse formulation presents a problem [48]. Instead, various simplifications need to be found.

Important modelling aspects [48]

- Modelling the uncertainty: Choosing the right process for interest rates, securities returns, liabilities
- External consistency: for all securities in the model, the expected present value of cash flows should be equal to the security's market price [7].
- Internal consistency: No implied arbitrage opportunities should be created by approximation errors or otherwise
- Transaction costs, taxes, regulatory aspects need to be taken into account appropriately

To that end, our study of relative performance of tree-based and ADDP-based solutions of a classical ALM problem formulation is highly relevant, especially since ADDP scales well with the number of random variables and with the number of scenarios, as we will show in the discussion.

7.1 Problem definition

In the following, an instance of an asset liability management problem as introduced by [33] is defined using the following constants and variables:

- $x_{i,t}^b$: units of security i bought at time t
- $x_{i,t}^s$: units of security i sold at time t
- $x_{i,t}^h$: holdings of security i **after** purchase/sell at time t
- $S_{i,t}$: ex-dividend price of security i at time t
- L_t : liability due at the end of period t
- $D_{i,t}$: dividend payment of security i at the end of period t
- P_t : discount factor = zero coupon bond
- y_t : single-period lending at the current short rate
- z_t : single-period borrowing at the current short rate + premium D
- k : interest rate spread between investor's rate and risk-free rate **annualized**
- c : transaction costs rate
- τ : end-of-period wealth utility factor
- $\overline{x_{i,0}}$: current holding of security i in the portfolio

Using these definitions, the asset liability management problem can be stated as follows:

$$\begin{aligned} \min \quad & (1+c) \sum_{i=1}^I S_{i,0} x_{i,0}^b - (1-c) \sum_{i=1}^I S_{i,0} x_{i,0}^s \\ & + P_0 y_0 - \exp(-k\Delta) P_0 z_0 - \tau y_T \end{aligned}$$

subject to

$$x_{i,0}^s - x_{i,0}^b + x_{i,0}^h = \overline{x_{i,0}} \quad \forall i \in I \quad (7.1)$$

$$x_{i,t-1}^h - x_{i,t}^s + x_{i,t}^b = x_{i,t}^h \quad \forall i \in I, t \in [1, \dots, T-1] \quad (7.2)$$

$$\begin{aligned} & \sum_{i=1}^I D_{i,t} x_{i,t-1}^h + y_{t-1} - z_{t-1} + (1-c) \sum_{i=1}^I S_{i,t} x_{i,t}^s \\ & - (1+c) \sum_{i=1}^I S_{i,t} x_{i,t}^b - P_t y_t + \exp(-k\Delta) P_t z_t = L_t \quad \forall t \in [1, \dots, T-1] \end{aligned} \quad (7.3)$$

$$\sum_{i=1}^I (D_{i,T} + S_{i,T}) x_{i,T-1}^h + y_{T-1} - z_{T-1} - y_T = L_T \quad (7.4)$$

$$x_{i,t}^b, x_{i,t}^s, x_{i,t}^h \geq 0 \quad \forall i \in I, t \in [0, \dots, T-1] \quad (7.5)$$

$$y_t \geq 0 \quad \forall t \in [0, \dots, T] \quad (7.6)$$

7.2 Parameters and models for the stochastic processes

Our comparison is done by running the rolling stochastic procedure for $N = 30$ iterations. The general parameters of the model are set as follows:

$$\begin{aligned} k &: 3\% \\ c &: 2\% \\ \tau &: 1 \\ \overline{x_{i,0}} &: 100 \text{ units of each security} \end{aligned}$$

First, the discount factor process P_t returns the price of a zero-coupon bond for every time step t , i.e. $P_t = \exp(-r_t \Delta)$, where r_t is the riskless one-period interest-rate. Modelling the interest rate process is a topic in itself with many developments taking place in the recent years. For a survey of methods, we refer to [16]. For our modelling purposes, we assume the interest rate to follow the Vasicek's model, which is one of the simplest meaningful choices. The interest rate is set to 2% in the beginning with no drift and a low σ_{r_t} of 0.01.

Furthermore, the securities processes of all i securities are assumed to follow a correlated multivariate GBM process, with a random correlation matrix sampled similarly to *chapter 5*. In particular, all securities processes exhibit no drift and the standard deviations are chosen in accordance with typical values in this domain, i.e. $\sigma_{S_i} \in [2\% - 5\%]$.

The liabilities process is assumed to follow an independent GBM process with low volatility, i.e. $\sigma_{L_t} = 0.03$

Lastly, the dividends D_i are set to a constant chosen randomly from the interval $[0.1\% - 3\%]$.

7.3 Numerical results

T	Profit	stderror	ProfitA	stderrorA	Diff %	t-paired test
3	1987.6	46.26	4244	44.48	53.1%	<1%
5	5678.3	133.16	8090.2	58.28	29.8%	<1%
10	9429.44	184.11	17557.7	86.422	46.3%	<1%
15	11352.68	214.7	27283.1	121.1	58.3%	<1%

Table 7.1: Comparison of payoffs, ALM with varying planning horizon

numAssets	Profit	stderror	ProfitA	stderrorA	Diff %	t-paired test
3	3581	127.1	6220.6	45.7	42.4%	<1%
5	2548.8	124.7	6035.4	43.668	57.7%	<1%
7	1895.1	142.52	5964.4	37.612	68.2%	<1%

Table 7.2: Comparison of payoffs, ALM with varying number of assets

The general observation here is that the gap between Profits generated by the tree-based method and Profits generated by ADDP (ProfitA) is clearly increasing with risking dimensionality. In particular, all our statistical tests showed that the sample ProfitA mean is significantly higher than the sample Profit mean. We also observed a decreasing gap for increasing variance of the stochastic processes. This can be explained by the fact that in general, stochastic planning for highly volatile processes is less effective. With $\sigma > 0.4$, no meaningful mean payoff could be constructed based on a sample size of $N = 30$.

σ_{S_i}	Profit	stderror	ProfitA	stderrorA	Diff %	t-paired test
0.2	3889.5	145.8	6213	68.8	37.4%	<1%
0.4	5870.7	531.2	7188.1	418.9	18.3%	<1%

Table 7.3: Comparison of payoffs, ALM with varying volatility of the processes

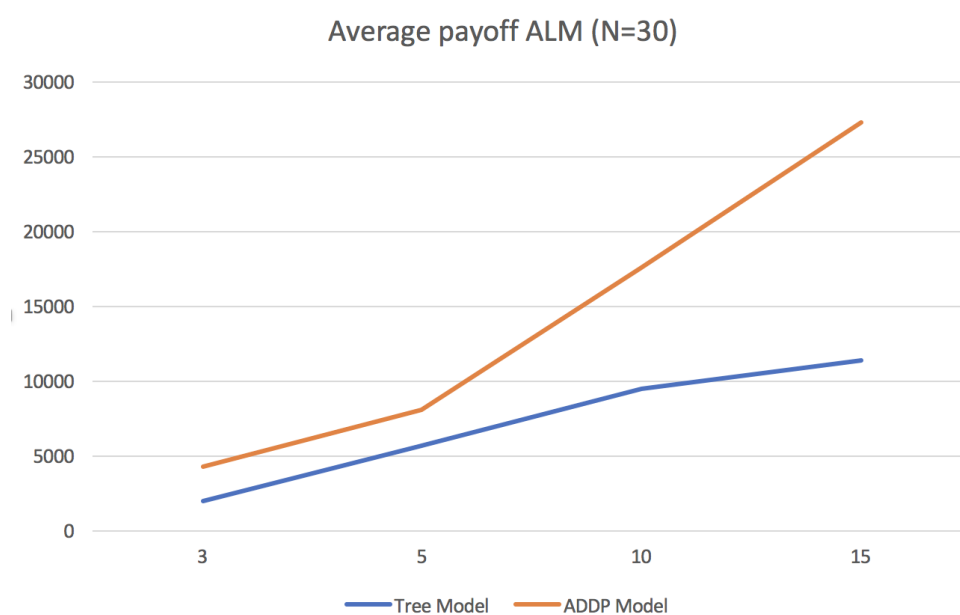


Figure 7.1: Payoffs comparison, ALM with varying planning horizon

Chapter 8

Conclusion and Outlook

We established a framework for comparing solutions of stochastic programs and applied it to study the difference between tree-based and ADDP-based methods. Moreover, we performed a general classifications of stochastic programs and aligned our study with it such that Based on our findings, the fact that policies generated by using ADDP significantly outperform those generated by tree-based methods can be confirmed for most problem instances. For smaller problems, we were not able to show a significant difference between the two approaches.

Apart from that, we observe that even bigger instances of SPs (e.g. 3 random variables, 50 planning periods) still remain tractable when using ADDP and can be solved in under 1 hour to find an optimal first-stage decision. For tree-based models, this was not the case. The performance of scenario reduction algorithm drops rapidly when the number of scenarios is increased beyond 10000 or when the number of stages is increased beyond 15 to 20. In general, while the construction of lattices and solving using ADDP scales well with the number of random variables and stages, the construction of trees quickly becomes infeasible for larger instances. By comparing the out-of-sample variance of decisions, we found out that ADDP-based ones showed less variance while yielding higher payoffs in high-dimensional case.

Outlook and future work:

Firstly, since ADDP is a relatively new method, there is a lot of room for algorithmic improvements.[35] In general, all features of Benders Decomposition could be applicable to ADDP as well, including

- Feasibility Cuts
- Regularization
- Stopping criteria
- Alternative forward passes
- Cut elimination
- Parallelization

Apart from that, an interesting and relevant research question would be to study the exact behavior of ADDP with respect to increasing dimension, i.e.: does the runtime scale linearly with respect to increasing dimension and is there a certain point after which the problems become completely intractable. In our studies, we found that going higher than $2e6$ scenarios can cause problems with lattice construction, but there is room for improvement upon this fairly imprecise rule-of-thumb result.

In addition to that, for the ALM model, our results show that a clear outperformance can be achieved by solving the LP using a lattice-based discretization and ADDP. Even for high number of securities (9+) and a high number of planning stages ($T = 15$), the runtime of one rolling planning instance remained well under 1 hour on a Core i7 computer with 16GB of RAM. This implies that the method could very well be applicable in practice. An interesting question would be whether it is possible to outperform the current state-of-the-art ALM models by dynamic planning with ADDP. The same idea applies for APP as well.

Lastly, it would be interesting to explore how other (coherent) risk measures can be combined with ADDP. In particular the combination of ADDP and nested CVaR is of interest for many practical purposes.

Chapter 9

Appendix

A Coding examples

Listing 9.1: Basic APP model (Chapter 5) formulation in Quasar/Matlab

```
1 %% declare objective and model
2 obj = quasarExpression();
3 model = quasarDecisionProblem('APP_M');
4
5 %% initial decision on labor capacity, constraints on capacity
6 labor_init = model.addVariable(0,'labor_init');
7 model.addConstraint(labor_init <= lab_max);
8 model.addConstraint(labor_init >= 0);
9
10 %% build up the model stage by stage
11 for t = 0:(stages-1)
12     overtime{t} = model.addVariable(t,'overtime');
13     produce{t} = model.addVariable(t, 'produce');
14     inventory{t} = model.addVariable(t, strcat('inv',num2str(i)));
15     model.addConstraint(inventory{t} <= max_inv);
16     %demand constraint
17     if(t>0)
18         model.addConstraint(produce+inventory{t-1}-inventory{t} >=
19             quasarRandomVariable('demand'));
20     else
21         model.addConstraint(produce-inventory{t} >= quasarRandomVariable('demand');
22
23     %% objective function
24     obj = obj + unit_costs*produce{t}+inv_costs*inventory{t};
25 end
26 %% add overtime hours to obj function
27 obj = obj+overtime*overtime_costs;
28 model.minimize(obj);
```

Listing 9.2: Basic APP model (Chapter 5) formulation in Quasar/Java

```

1 %% declare objective and model
2 Expression obj = new Expression();
3 DecisionProblem model = new DecisionProblem("APP_M");
4
5 %% initializing random variables
6 RandomVariable demand = new RandomVariable("demand");
7
8 %% initial decision on labor capacity, constraints on capacity
9 DecisionVariable labor_init = model.addVariable(0, 'labor_init').ub(lab_max);
10
11 %% build up the model stage by stage
12 DecisionVariable[] overtime = new DecisionVariable[stages];
13 DecisionVariable[] produce = new DecisionVariable[stages];
14 DecisionVariable[] inventory = new DecisionVariable[stages];
15 for(int t=0; t<stages-1; t++){
16     DecisionVariable overtime = model.addVariable(t, "overtime");
17     DecisionVariable produce = model.addVariable(t, "produce");
18     DecisionVariable inventory = model.addVariable(t, "inventory").ub(max_inv);
19     %% demand constraint
20     Constraint demand = model.addConstraint(produce[t].add(-1, inventory[t]).geq().
        add("demand"));
21     if(t>0)
22         demand.rhs().add(1, inventory[t-1]);
23
24     %% objective function
25     obj.add(unit_costs, produce).add(inv_costs, inventory[t]);
26 }
27 %% add overtime hours to obj function
28 obj = obj + overtime*overtime_costs;
29 model.minimize(obj);

```

B Generating NIG random variables

To generate random variables from the $NIG(\alpha, \beta, \mu, \delta)$ distribution, run [19]:

- Sample $u_1 \sim U[0, 1]$
- Let $Y = \Phi^{-1}(u_1) \sim N(0, 1)$
- Sample $u_2 \sim U[0, 1]$
- Let $V = (\Phi^{-1}(u_2))^2 \sim \chi^2_{(1)}$
- Set $\lambda = \delta^2, \psi = \frac{\delta}{\alpha^2 - \beta^2}$
- Compute $z_1 = \psi + \frac{\psi^2 V}{2\lambda} - \frac{\psi}{2\lambda} \sqrt{4\psi\lambda V + \psi^2 V^2}$
- Sample $u_3 \sim U[0, 1]$
- If $u_3 \leq \frac{\psi}{\psi + z_1}$, let $Z = z_1$, else let $Z = \frac{\psi^2}{z_1}$
- Return $X = \mu + \beta Z + \sqrt{Z}Y \sim NIG(\alpha, \beta, \mu, \delta)$

Bibliography

- [1] Siti Nazifah Zainol Abidin and Maheran Mohd Jaffar. A review on geometric brownian motion in forecasting the share prices in bursa malaysia. *World Applied Sciences Journal*, 17:87–93, 2012.
- [2] O. E. Barndorff-Nielsen and R. Stelzer. Absolute moments of generalized hyperbolic distributions and approximate scaling of normal inverse gaussian lévy processes. *Scandinavian Journal of Statistics*, 32(4):617–637, 2005.
- [3] J.F. Benders. Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4:238–252, 1962.
- [4] J.R. Birge and F. Louveaux. Multi-stage stochastic programs. In *Introduction to Stochastic Programming*, Springer Operations Research. 2011.
- [5] G. Bitran, E. Haas, and A Hax. Hierarchical production planning: A single-stage system. *Operations Research*, 1:717–743, 1981.
- [6] Svetlana Bizjajeva and Jimmy Olsson. Antithetic sampling for sequential monte carlo methods with application to state-space models. *Annals of the Institute of Statistical Mathematics*, 68(5):1025–1053, 2016.
- [7] Fischer Black, Emanuel Derman, and William Toy. A one-factor model of interest rates and its application to treasury bond options. *Financial analysts journal*, pages 33–39, 1990.
- [8] Marne C Cario and Barry L Nelson. Modeling and generating random vectors with arbitrary marginal distributions and correlation matrix. Technical report, Citeseer, 1997.
- [9] Hyeong Soo Chang, Michael C Fu, Jiaqiao Hu, Steven I Marcus, et al. A survey of some simulation-based algorithms for markov decision processes. *Communications in Information & Systems*, 7(1):59–92, 2007.
- [10] Zhuliang Chen and Peter A Forsyth. Implications of a regime-switching model on natural gas storage valuation and optimal operation. *Quantitative Finance*, 10(2): 159–176, 2010.
- [11] U Dieter and JH Ahrens. *Acceptance-rejection techniques for sampling from the gamma and beta distributions*. Stanford University. Department of Statistics, 1974.

- [12] Jitka Dupačová, Giorgio Consigli, and Stein W. Wallace. Scenarios for multistage stochastic programs. *Annals of Operations Research*, 100(1):25–53, 2000. ISSN 1572-9338. doi: 10.1023/A:1019206915174. URL <http://dx.doi.org/10.1023/A:1019206915174>.
- [13] Jitka Dupačová, Nicole Gröwe-Kuska, and Werner Römisch. Scenario reduction in stochastic programming. *Mathematical programming*, 95(3):493–511, 2003.
- [14] M. Dyer and L. Stougie. Computational complexity of stochastic programming problems. *Mathematical Programming Series A Springer*, 106:423–432, 2005.
- [15] M. Dyer and L. Stougie. Computational complexity of stochastic programming problems. *Mathematical Programming*, 106(3):423–432, 2006.
- [16] D Filipovic. Term-structure models: A graduate course, 2009.
- [17] S.M. Goldfeld and R.E. Quandt. The estimation of structural shifts by switching regressions. *Econometric Research Memorandum*, 147, 1973.
- [18] Stéphane Goutte, Benteng Zou, et al. Foreign exchange rates under markov regime switching model. Technical report, Center for Research in Economic Analysis, University of Luxembourg, 2011.
- [19] M. Groth. Simulation in financial mathematics. pages 33–33, 2005.
- [20] N. Gröwe-Kuska, H. Heitsch, and W. Römisch. Scenario reduction and scenario tree construction for power management problems. *IEEE Bologna Power Tech Conference*, 2003.
- [21] J.D. Hamilton. A new approach to the economic analysis of non-stationary time series. *Econometrica*, 57:357–384, 1989.
- [22] H. Heitsch and S. Dirkse. Scenario tree generation for stochastic programming models using gams/scenred. 2008. URL http://interfaces.gams-software.com/presentations/present_informs08_scenred.pdf.
- [23] Holger Heitsch and Werner Römisch. Scenario reduction algorithms in stochastic programming. *Computational optimization and applications*, 24(2-3):187–206, 2003.
- [24] Holger Heitsch, Werner Römisch, and Cyrille Strugarek. Stability of multistage stochastic programs. *SIAM Journal on Optimization*, 17(2):511–525, 2006.
- [25] J.L. Higle and S. Sen. *Stochastic Decomposition: A Statistical Method for Large Scale Stochastic Linear Programming*. Nonconvex Optimization and Its Applications. Springer US, 1996. ISBN 9780792338406. URL <https://books.google.de/books?id=4o5qWLXkK90C>.
- [26] Kjetil Høyland, Michal Kaut, and Stein W. Wallace. A heuristic for moment-matching scenario generation. *Computational Optimization and Applications*, 24(2):169–185, 2003. ISSN 1573-2894. doi: 10.1023/A:1021853807313. URL <http://dx.doi.org/10.1023/A:1021853807313>.

BIBLIOGRAPHY

- [27] Gerd Infanger. Monte carlo (importance) sampling within a benders decomposition algorithm for stochastic linear programs. *Annals of Operations Research*, 39(1):69–95, 1992. ISSN 1572-9338. doi: 10.1007/BF02060936. URL <http://dx.doi.org/10.1007/BF02060936>.
- [28] Farshid Jamshidian and Yu Zhu. Scenario simulation: Theory and methodology. *Finance and stochastics*, 1(1):43–67, 1996.
- [29] Harry Joe. Generating random correlation matrices based on partial correlations. *Journal of Multivariate Analysis*, 97(10):2177–2189, 2006.
- [30] Michal Kaut and Stein W. Wallace. Evaluation of scenario-generation methods for stochastic programming. *Pacific Journal of Optimization*, 3(2):257–271, 2007.
- [31] Eamonn Keogh and Abdullah Mueen. *Curse of Dimensionality*, pages 257–258. Springer US, Boston, MA, 2010. ISBN 978-0-387-30164-8. doi: 10.1007/978-0-387-30164-8_192. URL http://dx.doi.org/10.1007/978-0-387-30164-8_192.
- [32] John Frank Charles Kingman. *Poisson processes*. Wiley Online Library, 1993.
- [33] Pieter Klaassen. Financial asset-pricing theory and stochastic programming models for asset/liability management: A synthesis. *Management Science*, 44(1):31–48, 1998. ISSN 00251909, 15265501. URL <http://www.jstor.org/stable/2634425>.
- [34] D. Lewandowski, D. Kurowicka, and h Joe. Generating random correlation matrices based on vines and extended onion method. *Journal of Multivariate Analysis*, 100: 2177–2189, 2009.
- [35] N. Löhndorf and Wozabal. Optimal gas storage valuation and futures trading under a high-dimensional price process. Technical report, 2015.
- [36] N. Löhndorf, D. Wozabal, and S. Minner. Optimizing trading decisions for hydro storage systems using approximate dual dynamic programming. *Operation Research*, 61:810–823, 2013.
- [37] M. Loretan. Generating market risk scenarios using principal components analysis: methodological and practical considerations. *Federal Reserve Board*, 11, 1997.
- [38] F.V. Louveaux. A solution method for multistage stochastic programs with recourse with application to an energy investment problem. *Operations Research*, 28:889–902, 1980.
- [39] John M Mulvey and Hercules Vladimirov. Stochastic network optimization models for investment planning. *Annals of Operations Research*, 20(1):187–217, 1989.
- [40] M.V.F. Pereira and L.M.V.G. Pinto. Multi-stage stochastic optimization applied to energy planning. *Mathematical Programming*, 52(2):359–375, 1991.

- [41] Warren B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality (Wiley Series in Probability and Statistics)*. Wiley-Interscience, 2007. ISBN 0470171553.
- [42] W.B. Powell. Ai, or and control theory: A rosetta stone for stochastic optimization. 2012.
- [43] Werner Rmisch. Scenario reduction in stochastic programming: An approach using probability metrics. 2000.
- [44] Sheldon M Ross. *Introduction to probability models*. Academic press, 2014.
- [45] A. Shapiro and A. Nemirovski. On complexity of stochastic programming problems. In V. Jeyakumar and A. Rubinov, editors, *Continuous Optimization*, volume 99 of *Applied Optimization*, pages 111–146. 2005.
- [46] Alexander Shapiro. On a time consistency concept in risk averse multistage stochastic programming. *Operations Research Letters*, 37(3):143–147, 2009.
- [47] D.B. Shmoys and C. Swamy. An approximation scheme for stochastic linear programming and its application to stochastic integer programs. *J. ACM*, 53(6):978–1012, November 2006.
- [48] ManMohan S Sodhi. Lp modeling for asset-liability management: A survey of choices and simplifications. *Operations Research*, 53(2):181–196, 2005.
- [49] C. Swamy. Sampling-based approximation algorithms for multistage stochastic optimization. In *In Proceedings of the 46th Annual IEEE Symposium on Foundations of Computer Science*, pages 357–366, 2005.
- [50] Nikolas Topaloglou, Hercules Vladimirov, and Stavros A Zenios. Cvar models with selective hedging for international asset allocation. *Journal of Banking & Finance*, 26(7):1535–1561, 2002.
- [51] G. E. Uhlenbeck and L. S. Ornstein. On the theory of the brownian motion. *Phys. Rev.*, 36:823–841, Sep 1930. doi: 10.1103/PhysRev.36.823. URL <http://link.aps.org/doi/10.1103/PhysRev.36.823>.
- [52] R. Van Slyke and R.J-B. Wets. L-shaped linear programs with applications to optimal control and stochastic programming. *SIAM Journal on Applied Mathematics*, 12: 638–663, 1969.
- [53] Li yong Yu, Xiao dong Ji, and Shou yang Wang. Stochastic programming models in financial optimization: A survey.
- [54] Bowen Zhang, Lech Aleksander Grzelak, and Cornelis Willebrordus Oosterlee. Efficient pricing of commodity options with early-exercise under the ornstein–uhlenbeck process. *Applied Numerical Mathematics*, 62(2):91–111, 2012.

List of Tables

4.1	Case study stochastic programs	31
5.1	Costs for varying dependency between products	45
5.2	Costs for varying number of products, $T=5$, medium dependency	45
5.3	Costs for varying planning horizon length	45
6.1	Comparison of payoffs, basic flower girl model	51
6.2	Comparison of payoffs, random demand, random price flower girl model . .	51
6.3	Comparison of payoffs, CVaR variation, $\alpha = 0.01, \gamma = 0.5$	51
7.1	Comparison of payoffs, ALM with varying planning horizon	57
7.2	Comparison of payoffs, ALM with varying number of assets	57
7.3	Comparison of payoffs, ALM with varying volatility of the processes	57

List of Figures

2.1	Overview of solution measures	9
3.1	Ratio volume(hypersphere)/volume(hypercube),see [31]. As we can see, the ratio goes to zero for growing Dimension.	12
3.2	An exemplary scenario tree (a) modelling 11 scenarios and an exemplary scenario fan (b) modelling 8 scenarios.	13
3.3	An example of a scenario tree construction by forward reduction. Source: [22]	17
3.4	An exemplary scenario tree (a) and an exemplary scenario lattice (b) . . .	18
4.1	5 samples of the jump diffusion stochastic process	27
4.2	5 samples of the Ornstein-Uhlenbeck stochastic process	27
4.3	5 samples of the regime switching stochastic process	29
4.4	5 samples of the NIG stochastic process	30
4.5	degrees of correlation visualised	32
4.6	A schematic representation of the rolling horizon evaluation procedure . . .	34
4.7	A schematic representation of the out-of-sample rolling planning procedure	35
4.8	Overview: technical side of the rolling stochastic procedure	39
5.1	Payoff difference depending on T, numProd	46
5.2	Production decisions difference heatmap (T=5)	46
6.1	Comparison of payoffs, basic flower girl model	50
6.2	Comparison of payoffs, random demand and random price flower girl model	51
7.1	Payoffs comparison, ALM with varying planning horizon	58